VELLORE INSTITUTE OF TECHNOLOGY, VELLORE
School of Computer Science and Engineering

A Project Report on

ELIXOR: A Healthcare Management System

submitted for the course
BCSE307L Database Systems
Fall Semester 2026-27 (Assessments 8, 9 and 10)

Submitted by

S.No	Name	Reg. No
1	Sanidhya Srivastava	24BCB0135
2	Arna Adhikari	24BDS0363

Under the guidance of
Dr. Deepika J

Slot: L11+L12 Class No: VL2026270104071

Live: https://elixor-dun.vercel.app
GitHub: https://github.com/Zageraus/Elixo

[Month] 2026

ACKNOWLEDGEMENT

We would like to thank our faculty, Dr. Deepika J, for guiding us throughout this project. Her feedback during the first review helped us find mistakes in our database design and improve it, and we learned a lot in the process of fixing them.

We also thank the School of Computer Science and Engineering, Vellore Institute of Technology, for giving us the chance to do a full project as part of the Database Systems course. It helped us understand how the concepts taught in class are used in a real application.

Lastly, we thank our families and friends for their support while we were working on this project.

Sanidhya Srivastava (24BCB0135)
Arna Adhikari (24BDS0363)

ABSTRACT

ELIXOR is a healthcare management system made for small clinics. Using it, patients can book appointments with doctors, pay the consultation fee, save their health details like blood group and allergies, and view their prescriptions and medical reports online. Doctors can accept or decline appointments and write prescriptions. The admin can manage all the users and see statistics for the whole platform.

Since this is a Database Systems project, most of our work went into the database. We designed it in PostgreSQL with 13 tables, all normalized up to 3NF (and BCNF). Important rules are enforced by the database itself using primary and foreign keys, CHECK constraints, triggers, views and a stored procedure. The backend is written in Express.js and uses the Prisma ORM for all its queries, and whenever more than one table is changed, it is done inside a transaction. The frontend is made using Next.js. Along with this we implemented JWT based login, role based access for patients, doctors and admins, database backups using pg_dump, Docker images and a CI pipeline using GitHub Actions. The application is deployed, with the frontend on Vercel (https://elixor-dun.vercel.app) and the API on Render.

Keywords: DBMS, PostgreSQL, Normalization, Prisma ORM, Transactions, Triggers, Healthcare

CONTENTS

Section	Page
1. Problem Statement	5
2. Project Planning	6
3. Database Design	9
4. Database Implementation	16
5. Application Development	28
6. Authentication and Role-Based Authorization	37
7. Database Security	39
8. Database Backup and Recovery	41
9. Version Control	42
10. Deployment	44
11. Bonus	46
12. Project Execution	48
13. Improvements	51
14. Team Members' Commits	55
15. Challenges Faced	58
16. URLs, Tools and Resources	59
17. Conclusion and Future Work	60
References	61

1. Problem Statement

1.1 Background

In a lot of small clinics, appointments, prescriptions and patient records are still kept in paper registers, Excel sheets or WhatsApp chats. Because of this, some common problems come up:

- Two patients get booked with the same doctor at the same time.
- Lab reports get lost or are hard to find later.
- A prescription can be changed afterwards and there is no record of who changed it.
- The clinic has no easy way to know how much a doctor has earned or how busy they are.
- Anyone who has the file can see all the patients' data.

1.2 Existing System

Right now, small clinics mostly use manual records or general tools like spreadsheets. These have no relationships between the data, so the same patient's details are typed again and again in different places. There is also no validation, for example nothing stops someone from entering a negative fee or a medicine for zero days. There is no access control either, since anyone who opens the file can see or edit everything.

1.3 Proposed System

We propose ELIXOR, a web application built around a single relational database. Every patient, doctor, appointment, payment and prescription is stored once and linked to the others using foreign keys. Rules like "only one payment per appointment" or "a medicine has to be given for at least one day" are written as constraints, so the database will reject wrong data even if there is a bug in our code. Each user has to log in and can only see what their role allows.

We chose healthcare as our domain because the data has clear relationships (patients, doctors, visits, prescriptions, payments), and because correctness and privacy of data are very important in this field. This made it a good fit for the topics we studied in the course.

Fig. 1.1 shows the landing page of our application, which is also where users log in or create an account.

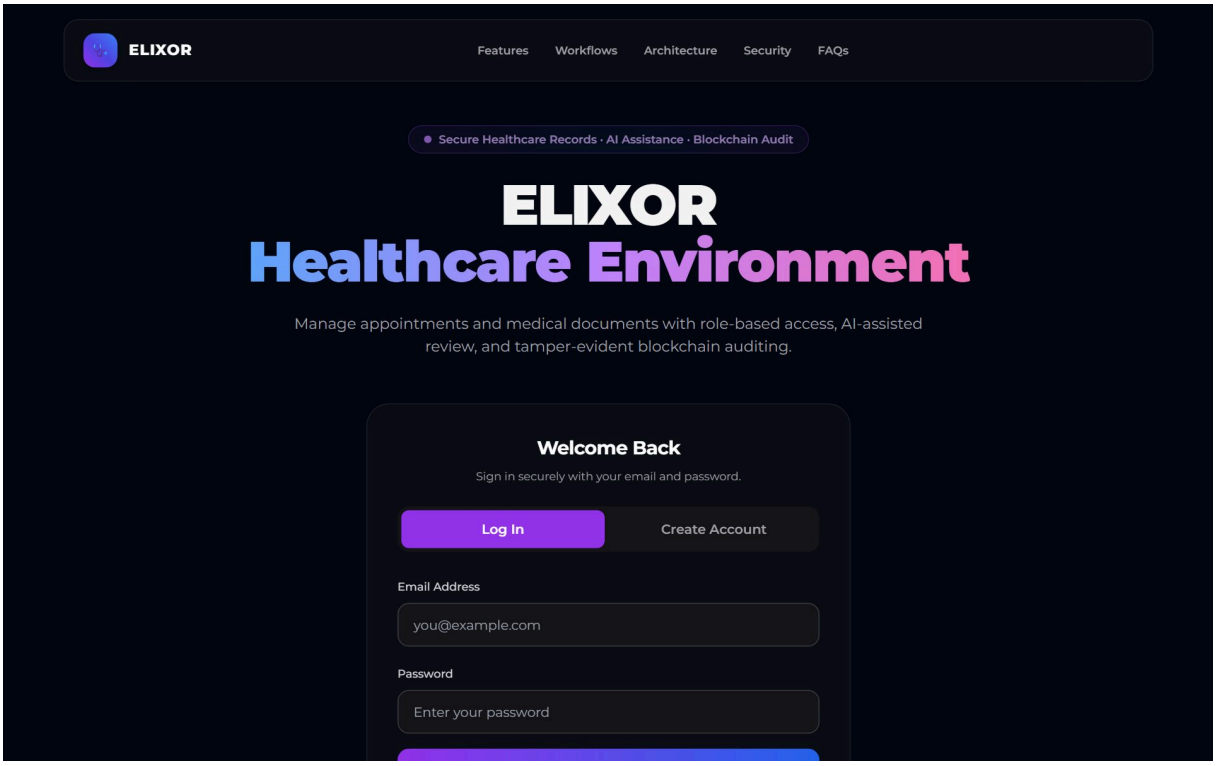


Fig. 1.1 Landing and login page of ELIXOR

2. Project Planning

2.1 Project Title

ELIXOR: A Healthcare Management System with a Normalized Relational Database, Role Based Access and Tamper Evident Medical Records.

2.2 Problem Statement

Small clinics keep appointments, prescriptions, payments and reports in registers and spreadsheets, which leads to double bookings, lost reports, untracked changes and no access control (Section 1). We need a relational database, with a web application on top of it, that stores this data once, enforces the clinic's rules, and lets each user see only what their role allows.

2.3 Objectives

1. To design an ER model and convert it into a relational schema in 3NF, with at least 4 main entities, 3 relationships and 8 tables.
2. To implement the schema in PostgreSQL with proper keys, constraints and indexes.
3. To use joins, aggregate queries, views, transactions, a stored procedure and triggers inside the actual application.

4. To access the database from the backend through an ORM (Prisma).
5. To build a responsive web app with login, dashboards, CRUD, search, filtering, pagination, validation and error handling.
6. To secure the system using JWT login, role based access with a single users table, hashed passwords and secrets kept out of the code.
7. To use GitHub for version control and to set up backups, Docker and deployment.

2.4 Scope

In scope: patient, doctor and admin accounts; booking and approving appointments; fee payment (simulated) with refund on cancellation; prescriptions with more than one medicine; uploading medical reports with an AI generated summary; notifications; admin statistics and user management; an audit log; and optionally saving the hash of reports and prescriptions on a blockchain so that any tampering can be detected.

Out of scope: a real payment gateway (payments are only simulated and we never ask for card details), giving medical advice, and hospital features like wards, pharmacy stock or insurance.

2.5 Team Members and Responsibilities

Name	Reg. No	Module	Responsibilities
Sanidhya Srivastava	24BCB0135	Database	ER diagram, relational schema and normalization; SQL migrations with keys, constraints, indexes, triggers, views, the stored procedure and row level security; seed data; Prisma ORM and transactions in the backend; backup and restore scripts
Arna Adhikari	24BDS0363	AI and Blockchain	AI summaries of medical reports; the MedicalAudit smart contract (Solidity, Hardhat); anchoring report and prescription hashes on the blockchain and checking them later for tampering (healthchain-upgrade branch, PR #1); running the AI summaries on Groq's free API for the deployed server

We divided the project into these two modules so that each of us could work on one part at a time. The web application (API, dashboards and login) was built on top of both modules.

2.6 Technology Stack

Layer	Technology
Frontend	Next.js 14 (React 18, TypeScript), Tailwind CSS

Backend	Node.js, Express.js 4, TypeScript, Zod for validation
ORM	Prisma 6
Database	PostgreSQL 17, hosted on Supabase
Authentication	JWT; passwords handled by Supabase Auth (bcrypt)
AI	Groq's free API (model openai/gpt-oss-120b) on the deployed server; Ollama (local model) when running on a laptop
Other	Hardhat and Solidity (blockchain), Docker, GitHub Actions
Hosting	Vercel for the frontend, Render for the API

To run the project, a computer needs Node.js 20 and npm, Git and a web browser. Any recent dual core processor with 8 GB RAM is enough (more helps if the local AI model is used). Docker, the PostgreSQL 17 client tools (for backup and restore) and Ollama (for local AI summaries) are optional.

2.7 Major Deliverables

- ER diagram, relational schema and data dictionary
- SQL migration files (001 to 012) and seed data
- Backend API and frontend web application
- Smart contract with tests
- Backup and restore scripts, Dockerfiles and CI workflow
- README and this report

2.8 Project Timeline

We planned the work in phases as shown in the Gantt chart below. The red line marks 29 October 2026.

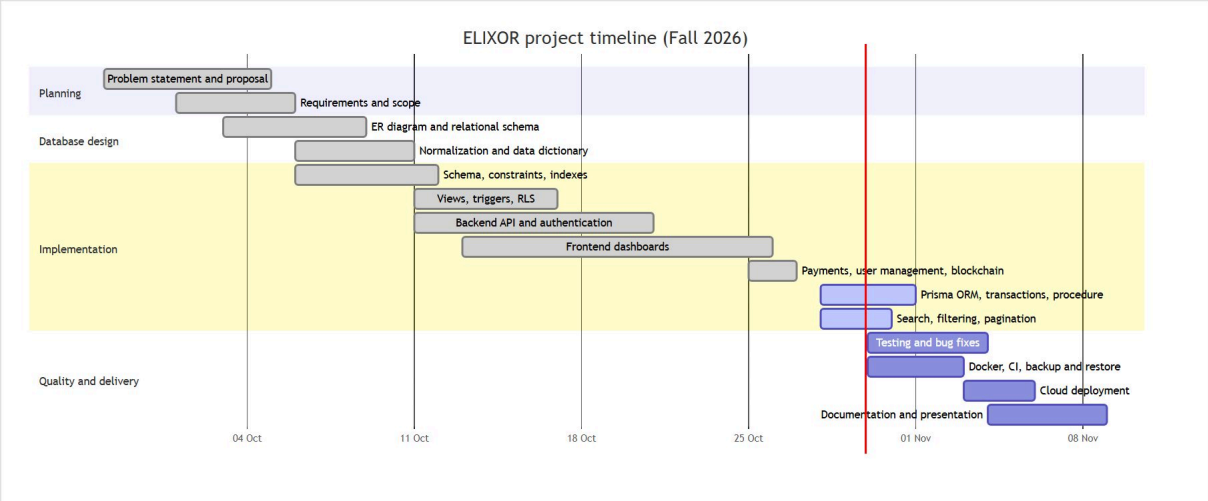


Fig. 2.1 Gantt chart of the project

3. Database Design

3.1 Identification of Entities

The five main entities in our system are **User**, **Appointment**, **Medical Report**, **Prescription** and **Payment**. Patients, doctors and admins are all stored as users, and their role decides what they are allowed to do. Some details only apply to one role, like a doctor's licence number and fee or a patient's blood group and allergies. We kept these in separate one-to-one profile tables, otherwise the users table would have many columns that are empty for most rows.

The supporting entities are the two profile tables, Prescription Item (a weak entity, one row for each medicine), AI Analysis, Blockchain Transaction, Notification, Audit Log and Auth Nonce (used for wallet login).

3.2 Relationships

Relationship	Cardinality
A patient books appointments	1 : N
A doctor attends appointments	1 : N
An appointment is paid by a payment	1 : 0..1
A doctor issues prescriptions to a patient	1 : N
An appointment can result in prescriptions	1 : N (optional)
A prescription contains medicines (prescription items)	1 : N (identifying)
A patient has medical reports; a doctor can review them	1 : N
A report is analysed by the AI	1 : N
A report or prescription is anchored by blockchain transactions	1 : N
A user has a doctor or patient profile	1 : 0..1
A user receives notifications and appears in the audit log	1 : N

3.3 ER Diagram

The ER diagram of our database is shown in Fig. 3.1. We have used crow's foot notation.

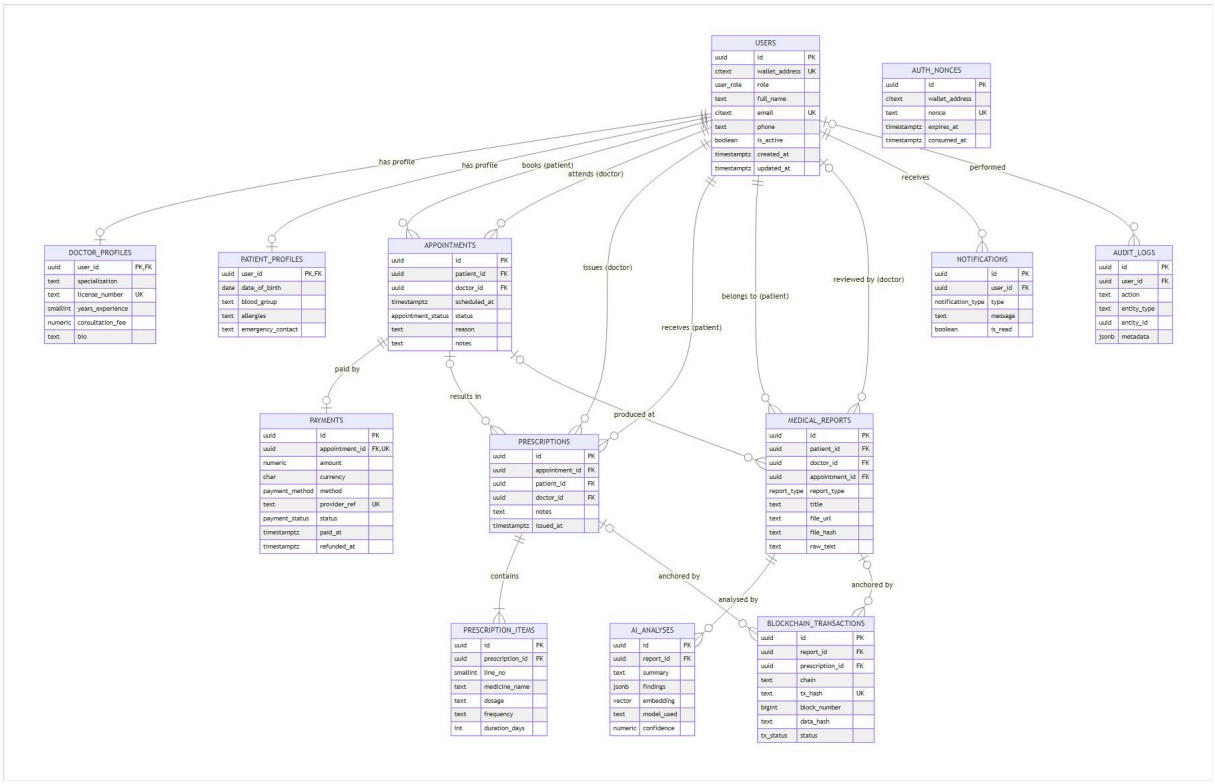


Fig. 3.1 ER diagram of ELIXOR

3.4 Relational Schema

To convert the ER diagram into tables, each entity became a table and each 1:N relationship became a foreign key on the "many" side. The 1:1 profile tables use the user's id as their primary key, and the weak entity Prescription Item uses the prescription's key along with a line number. In the schema below, primary keys are underlined and foreign keys are marked with *.

```
users (id, wallet_address, role, full_name, email, phone, is_active, created_at,
updated_at)
doctor_profiles (user_id*, specialization, license_number, years_experience,
consultation_fee, bio)
patient_profiles (user_id*, date_of_birth, blood_group, allergies, emergency_contact)
appointments (id, patient_id*, doctor_id*, scheduled_at, status, reason, notes,
created_at, updated_at)
payments (id, appointment_id*, amount, currency, method, provider, provider_ref,
status, paid_at, refunded_at)
prescriptions (id, appointment_id*, patient_id*, doctor_id*, notes, issued_at)
prescription_items (id, prescription_id*, line_no, medicine_name, dosage, frequency,
duration_days)
medical_reports (id, patient_id*, doctor_id*, appointment_id*, report_type, title,
file_url, file_hash, raw_text, created_at)
ai_analyses (id, report_id*, summary, findings, embedding, model_used, confidence,
created_at)
blockchain_transactions (id, report_id*, prescription_id*, chain, tx_hash,
block_number, data_hash, status, gas_used, created_at, confirmed_at)
notifications (id, user_id*, type, message, is_read, created_at)
audit_logs (id, user_id*, action, entity_type, entity_id, metadata, created_at)
auth_nonces (id, wallet_address, nonce, expires_at, consumed_at, created_at)
```

3.5 Data Dictionary

The data dictionary for the main tables is given below. The complete dictionary for all 13 tables is in `docs/DATABASE_DESIGN.md` in our repository.

users		
Column	**Type**	**Constraints / description**
id	uuid	Primary key, generated automatically
role	user_role (enum)	NOT NULL, default 'patient'; one of patient, doctor, admin
full_name	text	NOT NULL
email	citext	UNIQUE, case-insensitive
wallet_address	citext	UNIQUE, CHECK on the address format
is_active	boolean	NOT NULL, default true; false when an admin deactivates the user
created_at, updated_at	timestamptz	NOT NULL, default now(); updated_at kept current by a trigger

appointments		
Column	**Type**	**Constraints / description**
id	uuid	Primary key
patient_id, doctor_id	uuid	NOT NULL, foreign keys to users (ON DELETE CASCADE); CHECK patient_id <> doctor_id
scheduled_at	timestamptz	NOT NULL
status	appointment_status (enum)	NOT NULL, default 'requested'
reason, notes	text	Optional

payments		
Column	**Type**	**Constraints / description**
appointment_id	uuid	NOT NULL, UNIQUE (one payment per visit), foreign key to appointments
amount	numeric(10,2)	NOT NULL, CHECK amount > 0; the fee at the time of payment
method, status	enums	NOT NULL; status defaults to 'paid'
provider_ref	text	NOT NULL, UNIQUE receipt number
refunded_at	timestamptz	CHECK: set only when status is 'refunded'

prescription_items

Column	Type	Constraints / description
prescription_id	uuid	NOT NULL, foreign key to prescriptions (ON DELETE CASCADE)
line_no	smallint	NOT NULL, CHECK > 0, UNIQUE together with prescription_id
medicine_name	text	NOT NULL, CHECK not blank
dosage, frequency	text	NOT NULL
duration_days	integer	NOT NULL, CHECK > 0

3.6 Constraints

Other than primary and foreign keys, we have used NOT NULL on required columns, UNIQUE wherever duplicates should not be allowed, CHECK constraints for rules on values, and DEFAULT values for ids, timestamps and statuses. We ran the following query on our database to list all the CHECK and UNIQUE constraints:

Query:

```
SELECT conrelid::regclass::text as table_name, conname as constraint_name, case
contype when 'c' then 'CHECK' when 'u' then 'UNIQUE' end as type,
left(pg_get_constraintdef(oid), 70) as definition
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace and contype in ('c','u')
ORDER BY 1, 3, 2;
```

Output:

```
 table_name              | constraint_name                          | type   |
definition
-------------------------+------------------------------------------+--------+---------
----------------------------------------
 ai_analyses             | ai_analyses_confidence_check             | CHECK  | CHECK
(((confidence >= (0)::numeric) AND (confi…
 appointments            | chk_appt_roles                           | CHECK  | CHECK
((patient_id <> doctor_id))
 auth_nonces             | auth_nonces_nonce_key                    | UNIQUE | UNIQUE
(nonce)
 blockchain_transactions | chk_one_target                           | CHECK  | CHECK
((((report_id IS NOT NULL) AND (prescript…
 blockchain_transactions | blockchain_transactions_tx_hash_key      | UNIQUE | UNIQUE
(tx_hash)
 doctor_profiles         | doctor_profiles_consultation_fee_check | CHECK  | CHECK
((consultation_fee >= (0)::numeric))
 doctor_profiles         | doctor_profiles_years_experience_check | CHECK  | CHECK
((years_experience >= 0))
 doctor_profiles         | doctor_profiles_license_number_key     | UNIQUE | UNIQUE
(license_number)
 medical_reports         | chk_file_hash_format                     | CHECK  | CHECK
((file_hash ~* '^[a-f0-9]{64}$'::text))
 patient_profiles        | patient_profiles_blood_group_check     | CHECK  | CHECK
((blood_group = ANY (ARRAY['A+'::text, 'A…
 payments                | chk_refund_time                          | CHECK  | CHECK
(((status = 'refunded'::payment_status) =…
 payments                | payments_amount_check                    | CHECK  | CHECK
((amount > (0)::numeric))
 payments                | payments_appointment_id_key              | UNIQUE | UNIQUE
(appointment_id)
 payments                | payments_provider_ref_key                | UNIQUE | UNIQUE
(provider_ref)
 prescription_items      | prescription_items_duration_days_check | CHECK  | CHECK
((duration_days > 0))
 prescription_items      | prescription_items_line_no_check       | CHECK  | CHECK
((line_no > 0))
 prescription_items      | prescription_items_medicine_name_check | CHECK  | CHECK
((length(TRIM(BOTH FROM medicine_name)) >…
 prescription_items      | uq_prescription_line                     | UNIQUE | UNIQUE
(prescription_id, line_no)
 prescriptions           | chk_presc_roles                          | CHECK  | CHECK
((patient_id <> doctor_id))
 users                   | users_email_key                          | UNIQUE | UNIQUE
(email)
 users                   | users_wallet_address_key                 | UNIQUE | UNIQUE
(wallet_address)
(21 rows)
```

3.7 Normalization

Every table has a single column primary key, so there were no partial dependencies and 2NF was not a problem. However, when we reviewed our first design we found two problems, which we fixed in migration 012:

1. **1NF:** The prescriptions table stored all the medicines as a JSON array in one column. This is a repeating group, so it violates 1NF. We created a new table `prescription_items` which has one row for each medicine.

2. **3NF:** The payments table had `patient_id` and `doctor_id` columns. But every payment belongs to exactly one appointment, and the appointment already decides the patient and doctor (payment → appointment → patient). This is a transitive dependency, so we removed these two columns.

After these changes, the only determinants in each table are candidate keys (for example id, email, licence number, receipt number, or prescription_id + line_no). So the tables are also in BCNF. One thing we kept on purpose is that `prescriptions` still stores the patient and doctor, because a prescription does not always have an appointment. When an appointment is linked, a trigger checks that the patient and doctor match.

The output below shows the new prescription_items table, and the columns of payments after the change:

Query:

```
SELECT left(pi.prescription_id::text, 8) as prescription, pi.line_no,
pi.medicine_name, pi.dosage, pi.frequency, pi.duration_days
FROM prescription_items pi
  JOIN prescriptions pr on pr.id = pi.prescription_id
ORDER BY pr.issued_at desc, pi.line_no
LIMIT 10;
```

Output:

```
 prescription | line_no | medicine_name    | dosage | frequency
| duration_days
--------------+---------+------------------+--------+-------------------------------
+---------------
 9680a3e4     |       1 | Melatonon        | 500mg  | 2 times
|              7
 c5320db4     |       1 | Crocin           | 500mg  | Twice
|              7
 16734839     |       1 | Levothyroxine    | 50mcg  | once daily on an empty stomach
|             90
 f6cc65fa     |       1 | Atorvastatin     | 10mg   | once daily
|             90
 6e6f720f     |       1 | Atorvastatin     | 20mg   | once daily at night
|             90
 6e6f720f     |       2 | Aspirin          | 75mg   | once daily
|             90
 e8fedf84     |       1 | Paracetamol      | 650mg  | every 6 hours as needed
|              3
 3bb16db5     |       1 | Ibuprofen        | 400mg  | three times daily after food
|              5
 3bb16db5     |       2 | Thiocolchicoside | 4mg    | twice daily
|              5
 15583dee     |       1 | Escitalopram     | 5mg    | once daily
|             30
(10 rows)
```

Query:

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema='public' and table_name='payments'
ORDER BY ordinal_position;
```

Output:

```
 column_name    | data_type                | is_nullable
----------------+--------------------------+-------------
 id             | uuid                     | NO
 appointment_id | uuid                     | NO
 amount         | numeric                  | NO
 currency       | character                | NO
 method         | USER-DEFINED             | NO
 provider       | text                     | NO
 provider_ref   | text                     | NO
 status         | USER-DEFINED             | NO
 paid_at        | timestamp with time zone | NO
 refunded_at    | timestamp with time zone | YES
(10 rows)
```

3.8 Indexes

We created indexes on all foreign keys and on the columns that we filter or sort on (status, dates, role). The admin user search uses ILIKE '%text%', and a normal B-tree index does not help with that, so we added trigram (pg_trgm) GIN indexes on name and email.

Query:

```
SELECT tablename as table_name, indexname as index_name,
left(regexp_replace(indexdef, '^.* USING ', 'USING '), 60) as definition
FROM pg_indexes
WHERE schemaname = 'public' and indexname not like '%_pkey'
ORDER BY 1, 2;
```

Output:

```
 table_name              | index_name                           | definition
-------------------------+--------------------------------------+---------------------
----------------------------
 ai_analyses             | idx_ai_embedding                     | USING ivfflat
(embedding vector_cosine_ops) WIT…
 ai_analyses             | idx_ai_report                        | USING btree
(report_id)
 appointments            | idx_appt_doctor                      | USING btree
(doctor_id)
 appointments            | idx_appt_patient                     | USING btree
(patient_id)
 appointments            | idx_appt_scheduled                   | USING btree
(scheduled_at)
 appointments            | idx_appt_status                      | USING btree (status)
 audit_logs              | idx_audit_created                    | USING btree
(created_at)
 audit_logs              | idx_audit_entity                     | USING btree
(entity_type, entity_id)
 audit_logs              | idx_audit_user                       | USING btree
(user_id)
 auth_nonces             | auth_nonces_nonce_key                | USING btree (nonce)
 auth_nonces             | idx_auth_nonces_expiry               | USING btree
(expires_at) WHERE (consumed_at IS …
 auth_nonces             | idx_auth_nonces_wallet               | USING btree
(wallet_address)
 blockchain_transactions | blockchain_transactions_tx_hash_key | USING btree
(tx_hash)
 blockchain_transactions | idx_btx_prescription                 | USING btree
(prescription_id)
 blockchain_transactions | idx_btx_report                       | USING btree
(report_id)
 blockchain_transactions | idx_btx_status                       | USING btree (status)
 doctor_profiles         | doctor_profiles_license_number_key   | USING btree
(license_number)
 medical_reports         | idx_reports_created                  | USING btree
(created_at DESC)
 medical_reports         | idx_reports_doctor                   | USING btree
(doctor_id)
 medical_reports         | idx_reports_patient                  | USING btree
(patient_id)
 medical_reports         | idx_reports_type                     | USING btree
(report_type)
 notifications           | idx_notif_user                       | USING btree
(user_id, is_read)
 notifications           | idx_notif_user_created               | USING btree
(user_id, created_at DESC)
 payments                | idx_payments_status                  | USING btree (status)
 ... 12 more rows
(36 rows)
```

4. Database Implementation

4.1 Creating the Database

We used PostgreSQL 17 hosted on Supabase. The schema is created using SQL migration files that are kept in the repository, from `db/migrations/001_core_schema.sql` to `012_normalize_and_procedures.sql`. To set up a new database, `db/ALL_IN_ONE_SETUP.sql` runs all of them and then loads the seed data. In total the

database has 13 tables, 13 primary keys and 17 foreign keys. The number of rows in each table at the time of writing is:

Table	Rows	Table	Rows
users	21	medical_reports	11
doctor_profiles	10	ai_analyses	9
patient_profiles	9	blockchain_transactions	21
appointments	32	notifications	66
payments	11	audit_logs	128
prescriptions	10	auth_nonces	0
prescription_items	14		

[Insert screenshot: Supabase dashboard, Table Editor showing the tables in the public schema]

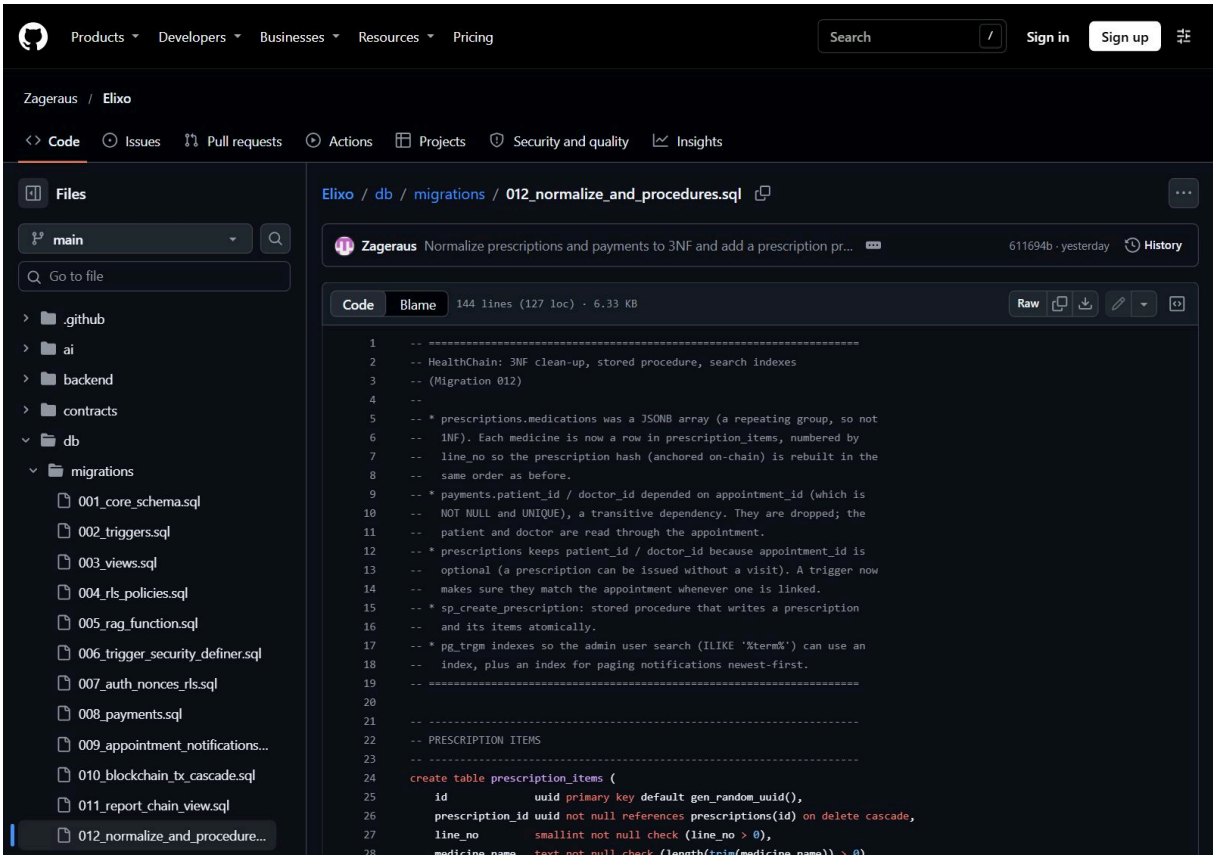


Fig. 4.1 Migration 012 in our repository

4.2 CRUD Operations

To show all four operations, we created a test patient using the API, read it back, updated it, deactivated it and then deleted it. This account was made only for testing.

Create

Request:

```
POST /api/auth/register

{
  "email": "crud.demo.1793288827949@example.com",
  "password": "••••••••••",
  "full_name": "CRUD Demo Patient",
  "role": "patient"
}
```

Response:

```
Status: 201
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5…(JWT)",
  "user": {
    "id": "eb0c8807-df9f-46a9-b299-b54e62b6afc5",
    "role": "patient",
    "full_name": "CRUD Demo Patient",
    "email": "crud.demo.1793288827949@example.com",
    "wallet_address": ""
  }
}
```

Read

Request:

```
GET /api/auth/me
```

Response:

```
Status: 200
{
  "id": "eb0c8807-df9f-46a9-b299-b54e62b6afc5",
  "role": "patient",
  "full_name": "CRUD Demo Patient",
  "email": "crud.demo.1793288827949@example.com",
  "phone": null,
  "is_active": true,
  "created_at": "2026-10-29T15:47:09.617Z",
  "wallet_address": "",
  "doctor_profile": null
}
```

Update

Request:

```
PUT /api/patients/me/profile

{
  "date_of_birth": "2004-05-17",
  "blood_group": "B+",
  "allergies": "Dust, sulfa drugs",
  "emergency_contact": "Parent 9800000000"
}
```

Response:

```
Status: 200
{
  "date_of_birth": "2004-05-17",
  "blood_group": "B+",
  "allergies": "Dust, sulfa drugs",
  "emergency_contact": "Parent 9800000000"
}
```

Request:

```
PATCH /api/admin/users/eb0c8807-df9f-46a9-b299-b54e62b6afc5/active

{
  "is_active": false
}
```

Response:

```
Status: 200
{
  "id": "eb0c8807-df9f-46a9-b299-b54e62b6afc5",
  "is_active": false,
  "cancelled_appointments": 0,
  "patients_notified": 0
}
```

Delete

Request:

```
DELETE /api/admin/users/eb0c8807-df9f-46a9-b299-b54e62b6afc5
```

Response:

```
Status: 200
{
  "id": "eb0c8807-df9f-46a9-b299-b54e62b6afc5",
  "deleted": true,
  "patients_notified": 0
}
```

Request:

```
GET /api/admin/users?role=patient&q=crud%20demo
```

Response:

```
Status: 200
{
  "items": [],
  "total": 0,
  "page": 1,
  "page_size": 10,
  "total_pages": 1,
  "role_totals": {
    "patient": 0,
    "doctor": 0
  }
}
```

The other tables work the same way through their own endpoints, for example creating and updating appointments, creating prescriptions, uploading reports and marking notifications as read.

4.3 Joins

The query below lists the recent appointments along with the patient's name, the doctor's name, the doctor's specialization and whether the visit is paid. The users table is joined twice here, once for the patient and once for the doctor.

Query:

```
SELECT p.full_name as patient, d.full_name as doctor, dp.specialization,
to_char(a.scheduled_at at time zone 'Asia/Kolkata','DD Mon YYYY HH24:MI') as
scheduled_ist, a.status::text as status, coalesce(pay.status::text, '-') as payment
FROM appointments a
  JOIN users p on p.id = a.patient_id
  JOIN users d on d.id = a.doctor_id
  LEFT JOIN doctor_profiles dp on dp.user_id = d.id
  LEFT JOIN payments pay on pay.appointment_id = a.id
WHERE a.scheduled_at between '2026-09-01' and '2026-12-31'
ORDER BY a.scheduled_at desc
LIMIT 8;
```

Output:

```
 patient                 | doctor              | specialization          |
scheduled_ist     | status    | payment
-------------------------+---------------------+-------------------------+-----------
--------+-----------+---------
 Arna Adhikari 24BDS0363 | Dr. Ananya Iyer     | Dermatology             | 12 Dec
2026 12:12 | requested | -
 Arna Adhikari 24BDS0363 | Dr. Ananya Iyer     | Dermatology             | 07 Nov
2026 16:15 | requested | -
 Arna Adhikari 24BDS0363 | Sanidhya Srivastava | General Medicine        | 07 Nov
2026 16:13 | requested | -
 Arna Adhikari 24BDS0363 | Dr. Ananya Iyer     | Dermatology             | 07 Nov
2026 16:13 | requested | -
 Sneha Kulkarni          | Dr. Kavya Reddy     | Gynecology & Obstetrics | 05 Nov
2026 10:00 | confirmed | paid
 Chitra Menon            | Dr. Priya Sharma    | Pediatrics              | 04 Nov
2026 15:30 | requested | -
 Aarav Patel             | Dr. Rohan Mehta     | Orthopedics             | 03 Nov
2026 16:00 | requested | -
 Ishita Banerjee         | Sanidhya Srivastava | General Medicine        | 02 Nov
2026 09:30 | requested | -
(8 rows)
```

4.4 Aggregate Queries

This query finds the number of appointments, completed visits and total revenue for each doctor. It uses GROUP BY, HAVING and aggregates with FILTER:

Query:

```
SELECT d.full_name as doctor, dp.specialization, count(a.id) as appointments,
count(*) filter (where a.status = 'completed') as completed, coalesce(sum(pay.amount)
filter (where pay.status = 'paid'), 0) as revenue_inr
FROM users d
  JOIN doctor_profiles dp on dp.user_id = d.id
  JOIN appointments a on a.doctor_id = d.id
  LEFT JOIN payments pay on pay.appointment_id = a.id
GROUP BY d.full_name, dp.specialization
HAVING count(a.id) >= 2
ORDER BY revenue_inr desc, appointments desc;
```

Output:

```
 doctor              | specialization          | appointments | completed |
revenue_inr
--------------------+-------------------------+--------------+-----------+----------
---
 Dr. Meera Joshi     | Psychiatry              |            2 |         1 |
2600
 Dr. Vikram Singh    | Endocrinology           |            3 |         2 |
2200
 Dr. Kavya Reddy     | Gynecology & Obstetrics |            2 |         1 |
2000
 Dr. Rohan Mehta     | Orthopedics             |            2 |         1 |
1200
 Dr. Ananya Iyer     | Dermatology             |            7 |         1 |
800
 Dr. Priya Sharma    | Pediatrics              |            2 |         0 |
700
 Sanidhya Srivastava | General Medicine        |            6 |         2 |
500
 Dr. Bob Rao         | Cardiology              |            5 |         1 |
500
 Dr. Arjun Nair      | Neurology               |            2 |         0 |
0
(9 rows)
```

4.5 Views

We created five views. `v_patient_dashboard` and `v_doctor_dashboard` are used for the appointment lists, `v_report_with_analysis` joins each report with its latest AI summary, `v_admin_analytics` calculates the numbers shown on the admin dashboard, and `v_doctor_weekly_load` counts the appointments of each doctor per week.

Query:

```
SELECT table_name as view_name
FROM information_schema.views
WHERE table_schema = 'public'
ORDER BY 1;
```

Output:

```
 view_name
-----------------------
 v_admin_analytics
 v_doctor_dashboard
 v_doctor_weekly_load
 v_patient_dashboard
 v_report_with_analysis
(5 rows)
```

Query:

```
SELECT u.full_name as doctor, to_char(l.week_start,'DD Mon YYYY') as week,
l.appointment_count, l.completed_count, l.cancelled_count
FROM v_doctor_weekly_load l
  JOIN users u on u.id = l.doctor_id
ORDER BY l.week_start desc, u.full_name
LIMIT 8;
```

Output:

```
 doctor               | week        | appointment_count | completed_count |
cancelled_count
---------------------+-------------+-------------------+-----------------+-----------
------
 Dr. Ananya Iyer     | 25 Feb 2222 |                 1 |               0 |
0
 Dr. Bob Rao         | 25 Feb 2222 |                 1 |               0 |
0
 Dr. Bob Rao         | 18 Feb 2222 |                 1 |               0 |
0
 Sanidhya Srivastava | 18 Feb 2222 |                 1 |               0 |
0
 Dr. Ananya Iyer     | 07 Dec 2026 |                 1 |               0 |
0
 Dr. Ananya Iyer     | 02 Nov 2026 |                 2 |               0 |
0
 Dr. Kavya Reddy     | 02 Nov 2026 |                 1 |               0 |
0
 Dr. Priya Sharma    | 02 Nov 2026 |                 1 |               0 |
0
(8 rows)
```

4.6 Transactions

Any operation that changes more than one row is run inside a transaction (`prisma.$transaction`), so either the whole thing succeeds or nothing changes. We used transactions for registering a doctor (user and profile), making a payment, issuing a prescription, deactivating or deleting a user, and changing the status of an appointment. In the payment transaction we also lock the appointment row using SELECT ... FOR UPDATE, so if two payment requests come at the same time, only one of them can go through.

The example below shows a rollback. One of the medicines has a duration of 0 days, which fails a CHECK constraint, so the whole prescription is rolled back, including the prescription row that had already been inserted.

```
SELECT count(*) FROM prescriptions
10

BEGIN; CALL sp_create_prescription(... items: Paracetamol (3 days), Bad line (0 days)
...)
ERROR: new row for relation "prescription_items" violates check constraint
"prescription_items_duration_days_check"

(transaction aborted)
ROLLBACK

SELECT count(*) FROM prescriptions
10   (unchanged: the header row was rolled back with the bad item)
```

Fig. 4.2 shows our payment route, where the whole checkout runs in one transaction along with the row lock.

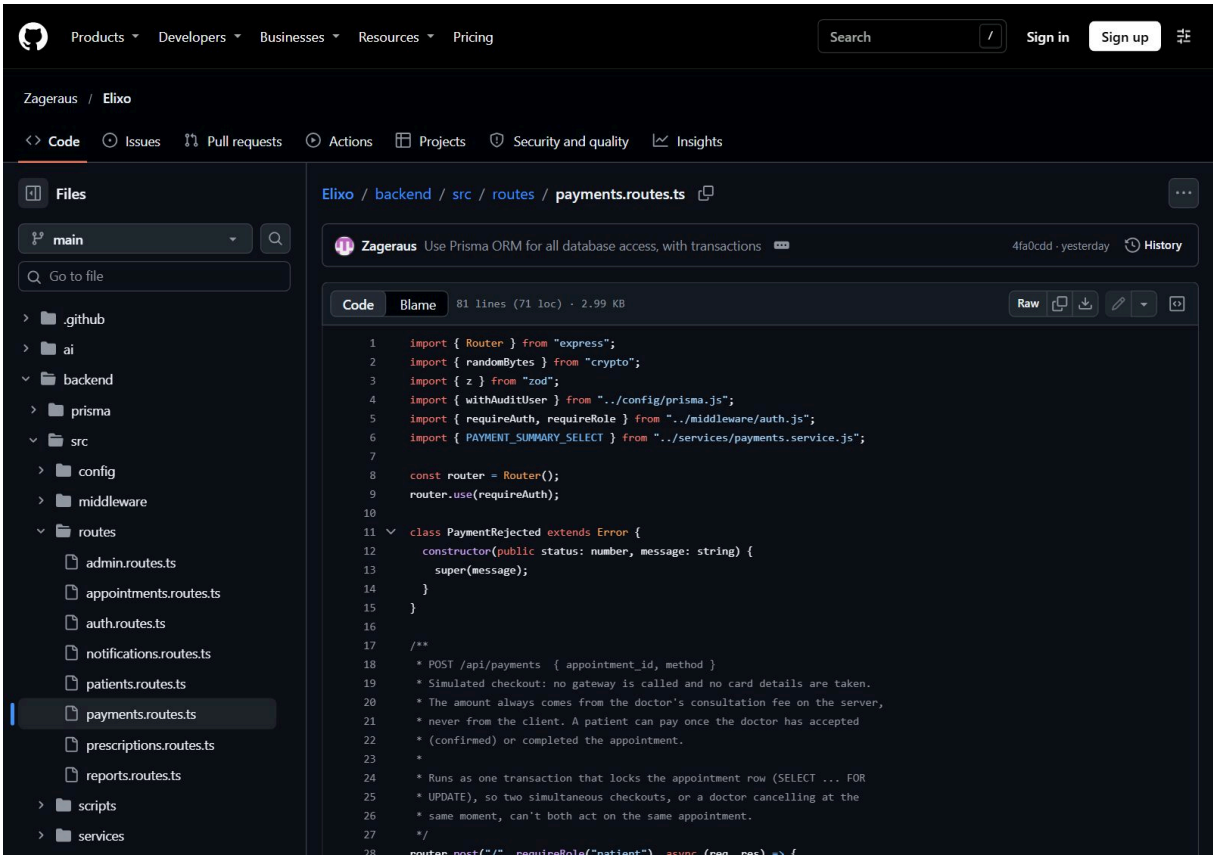


Fig. 4.2 Payment route using a transaction and a row lock

4.7 Stored Procedure

`sp_create_prescription` takes the doctor, the patient, an optional appointment, notes and the list of medicines in JSON format. It checks the input, inserts the prescription and one row for each medicine, and returns the id of the new prescription through an INOUT parameter. The backend calls this procedure whenever a doctor issues a prescription. We tested it inside a transaction and rolled it back after:

```
BEGIN

CALL sp_create_prescription(doctor, patient, NULL, 'Demo for report', '[Amoxicillin,
ORS]', NULL)
p_prescription_id = d1c8508b-e0f0-43a5-9d3f-393f88eb8679

SELECT line_no, medicine_name, dosage, duration_days FROM prescription_items WHERE
prescription_id = <new id>
1 | Amoxicillin | 500mg | 5
2 | ORS | 1 sachet | 2

ROLLBACK
demo data discarded
```

Query:

```
SELECT p.proname as name, case p.prokind when 'p' then 'PROCEDURE' else 'FUNCTION'
end as kind, pg_get_function_identity_arguments(p.oid) as arguments
FROM pg_proc p
WHERE p.pronamespace = 'public'::regnamespace and (p.prokind = 'p' or p.proname like
'fn\_%' or p.proname like 'match\_%')
ORDER BY 2 desc, 1;
```

Output:

```
 name                               | kind      | arguments
------------------------------------+-----------+-------------------------------------
-------------
 sp_create_prescription             | PROCEDURE | IN p_doctor_id uuid, IN p_patient_id
uuid, IN p…
 fn_audit_trigger                   | FUNCTION  |
 fn_check_doctor_role               | FUNCTION  |
 fn_check_prescription_appointment  | FUNCTION  |
 fn_current_user_id                 | FUNCTION  |
 fn_current_user_role               | FUNCTION  |
 fn_notify_appointment_change       | FUNCTION  |
 fn_notify_new_prescription         | FUNCTION  |
 fn_notify_new_report               | FUNCTION  |
 fn_refund_on_cancel                | FUNCTION  |
 fn_set_updated_at                  | FUNCTION  |
 match_patient_analyses             | FUNCTION  | p_patient_id uuid, p_query_embedding
vector, p_…
(12 rows)
```

4.8 Triggers

There are 13 triggers in our database. They keep `updated_at` up to date, write every change on the main tables into `audit_logs`, send notifications, check that the doctor and patient of an appointment have the correct roles, refund the payment when an appointment is cancelled, and keep prescriptions consistent with their appointment.

Query:

```
SELECT event_object_table as table_name, trigger_name, string_agg(event_manipulation,
' / '
ORDER BY event_manipulation) as events, action_timing as timing
FROM information_schema.triggers
WHERE trigger_schema = 'public'
GROUP BY 1, 2, 4
ORDER BY 1, 2;
```

Output:

```
 table_name      | trigger_name                          | events                    |
timing
-----------------+---------------------------------------+---------------------------+---
-----
 appointments    | trg_appointments_updated_at           | UPDATE                    |
BEFORE
 appointments    | trg_audit_appointments                | DELETE / INSERT / UPDATE |
AFTER
 appointments    | trg_check_appt_roles                  | INSERT / UPDATE           |
BEFORE
 appointments    | trg_notify_appointment                | INSERT / UPDATE           |
AFTER
 appointments    | trg_refund_on_cancel                  | UPDATE                    |
AFTER
 medical_reports | trg_audit_medical_reports             | DELETE / INSERT / UPDATE |
AFTER
 medical_reports | trg_notify_report                     | INSERT                    |
AFTER
 payments        | trg_audit_payments                    | DELETE / INSERT / UPDATE |
AFTER
 prescriptions   | trg_audit_prescriptions               | DELETE / INSERT / UPDATE |
AFTER
 prescriptions   | trg_check_prescription_appointment | INSERT / UPDATE           |
BEFORE
 prescriptions   | trg_notify_prescription               | INSERT                    |
AFTER
 users           | trg_audit_users_role                  | UPDATE                    |
AFTER
 users           | trg_users_updated_at                  | UPDATE                    |
BEFORE
(13 rows)
```

Below we tested the refund trigger. A paid appointment is cancelled and the payment status changes to refunded on its own. The audit log also shows which user made the change. This was also rolled back after the test.

```
BEGIN; SELECT set_config('app.current_user_id', <admin id>, true)
acting user recorded for the audit trigger

SELECT status, payment FROM appointment of Sneha Kulkarni
confirmed | paid

UPDATE appointments SET status = 'cancelled' WHERE id = <that appointment>
UPDATE 1   -> trg_refund_on_cancel, trg_notify_appointment, trg_audit_appointments
fire

SELECT status, refunded_at IS NOT NULL FROM payments WHERE appointment_id = <it>
refunded | true

SELECT action, entity_type, acted_by FROM audit_logs (newest for it)
UPDATE | appointments | Alice Admin

ROLLBACK
demo discarded: appointment and payment are unchanged
```

4.9 ORM (Prisma)

Our backend connects to the database through Prisma. Each table has a model in `backend/prisma/schema.prisma`. We used raw SQL only where Prisma does not directly

support something (reading views, calling the stored procedure and the row lock), and even there we used Prisma's `$queryRaw`, which uses parameters.

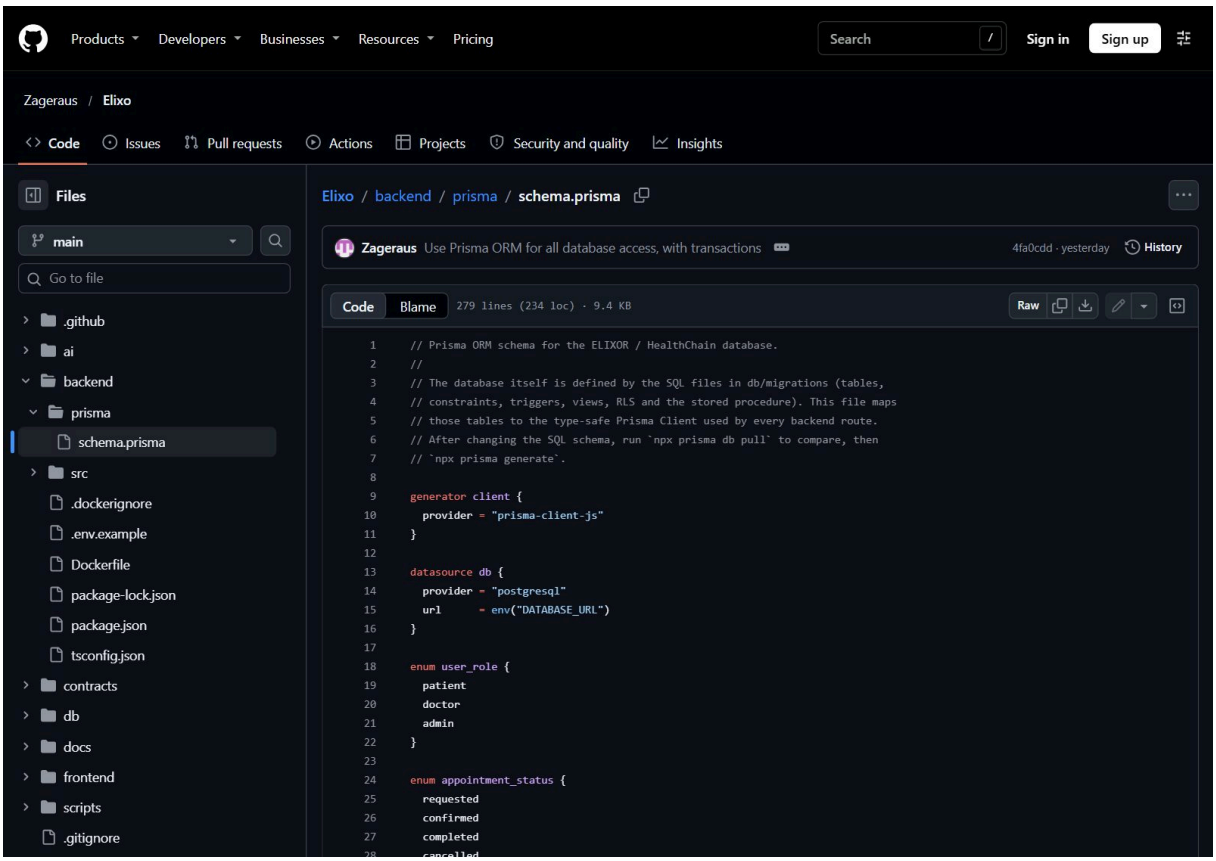


Fig. 4.3 Part of our Prisma schema

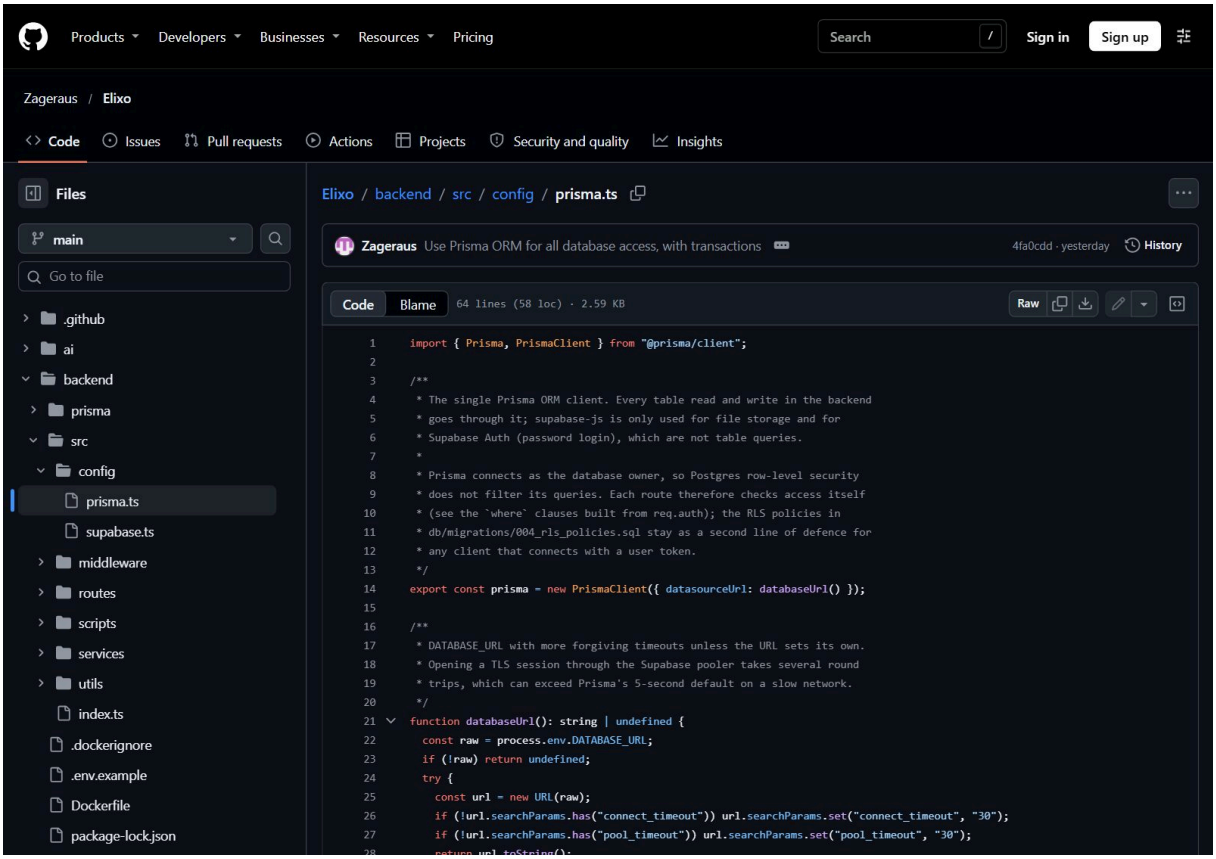


Fig. 4.4 Prisma client setup and the transaction helper

[Insert screenshot: Supabase SQL Editor running a query, for example select * from v_admin_analytics]

5. Application Development

The frontend of ELIXOR is a Next.js application and the backend is a REST API made with Express.js. There are three types of users and each one has a separate dashboard. In this section we show the main screens of the application.

5.1 Login and Registration

Fig. 5.1 shows the registration form. If the user enters a password that is too short, a validation message is shown and the form is not submitted.

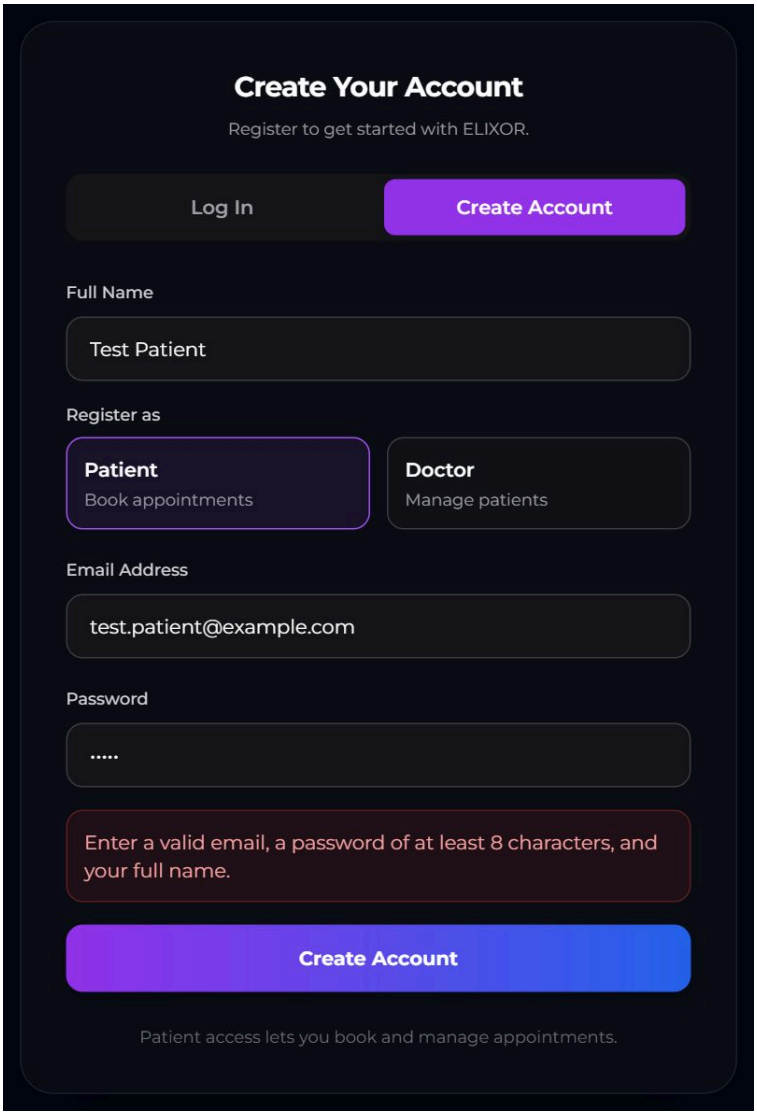


Fig. 5.1 Registration form with a validation message

5.2 Dashboards

The patient dashboard shows upcoming appointments, prescriptions, reports and payments. The doctor dashboard shows the appointment queue and the prescription form, and the admin dashboard shows the statistics and user management.

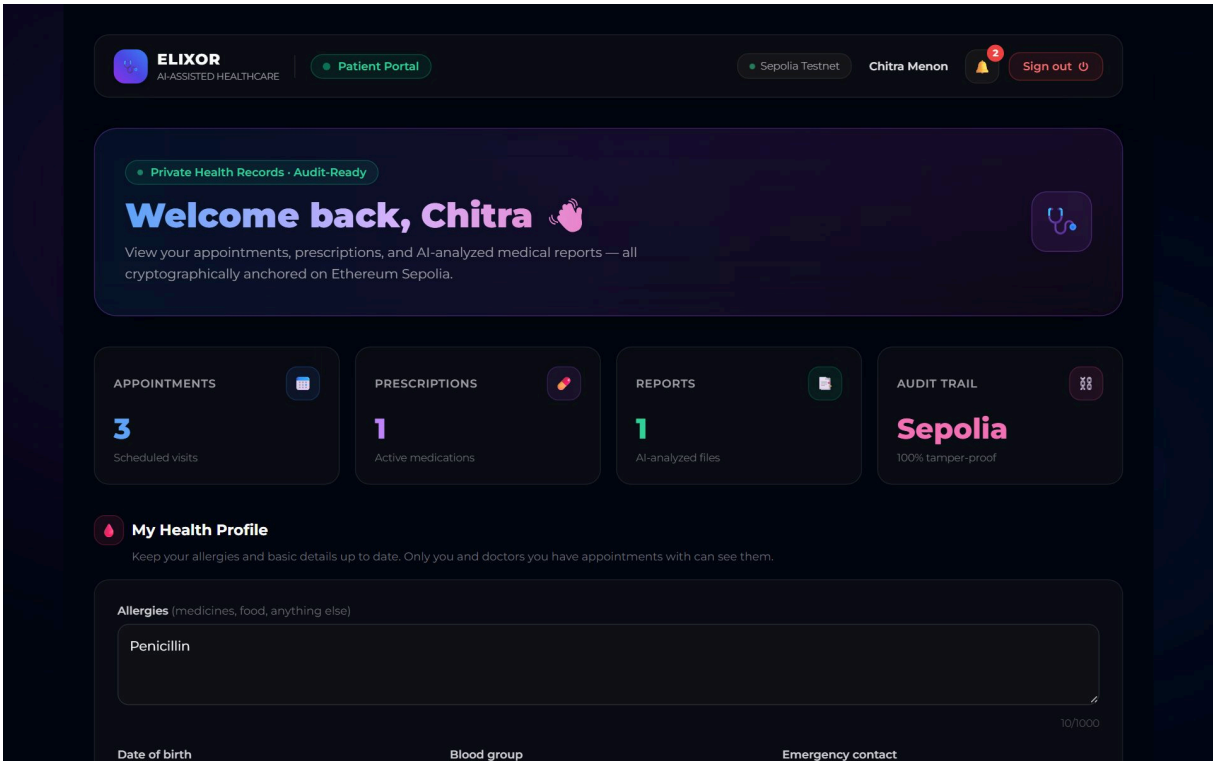


Fig. 5.2 Patient dashboard

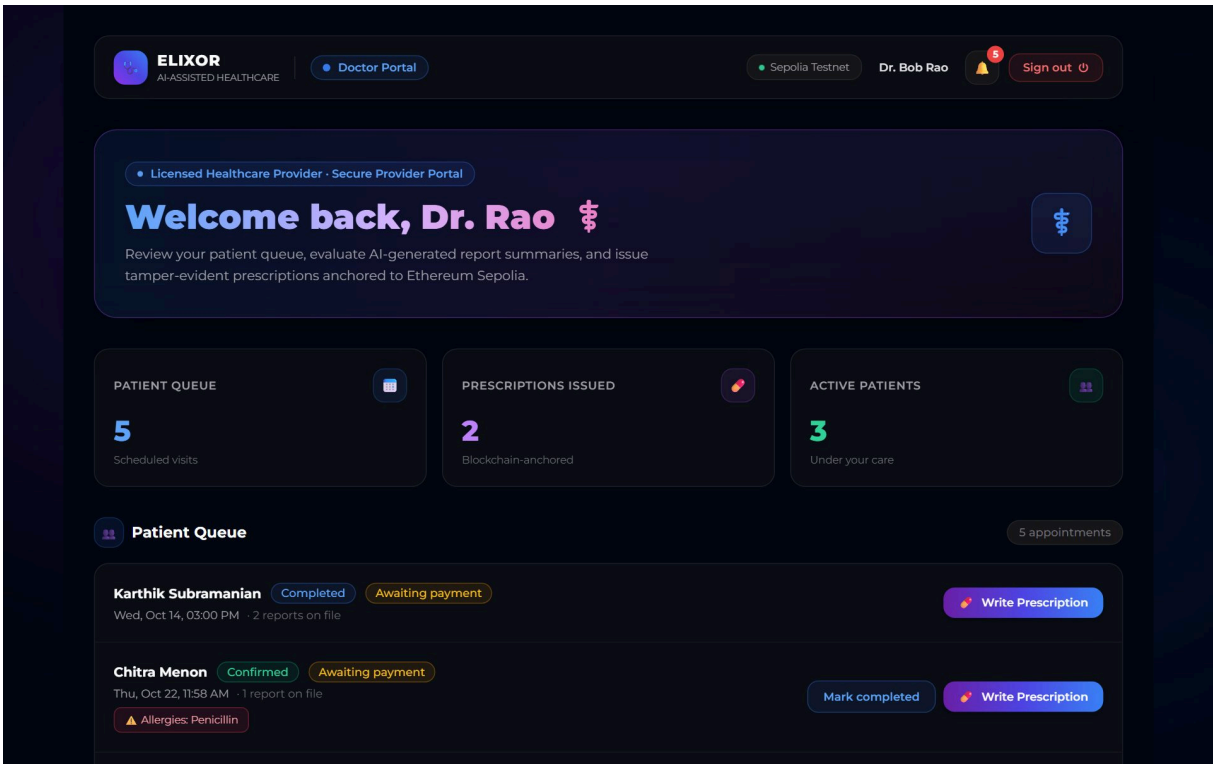


Fig. 5.3 Doctor dashboard

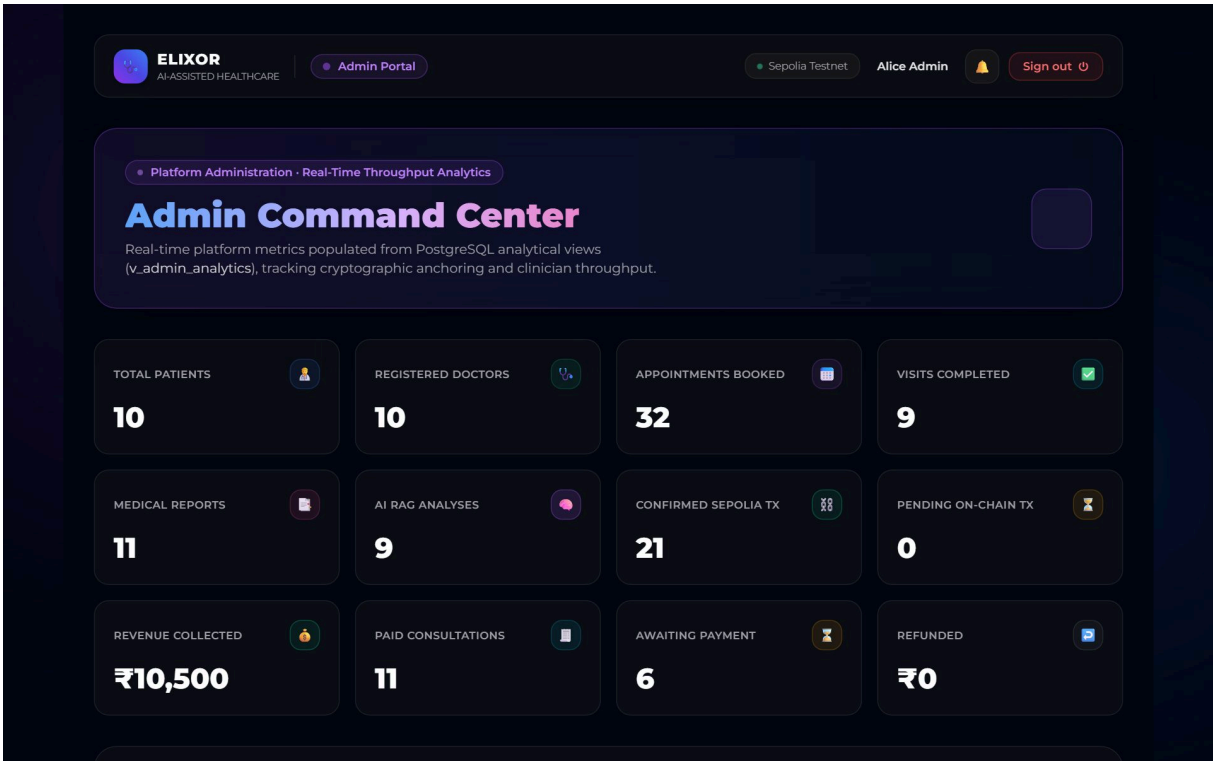


Fig. 5.4 Admin dashboard

5.3 CRUD from the Interface

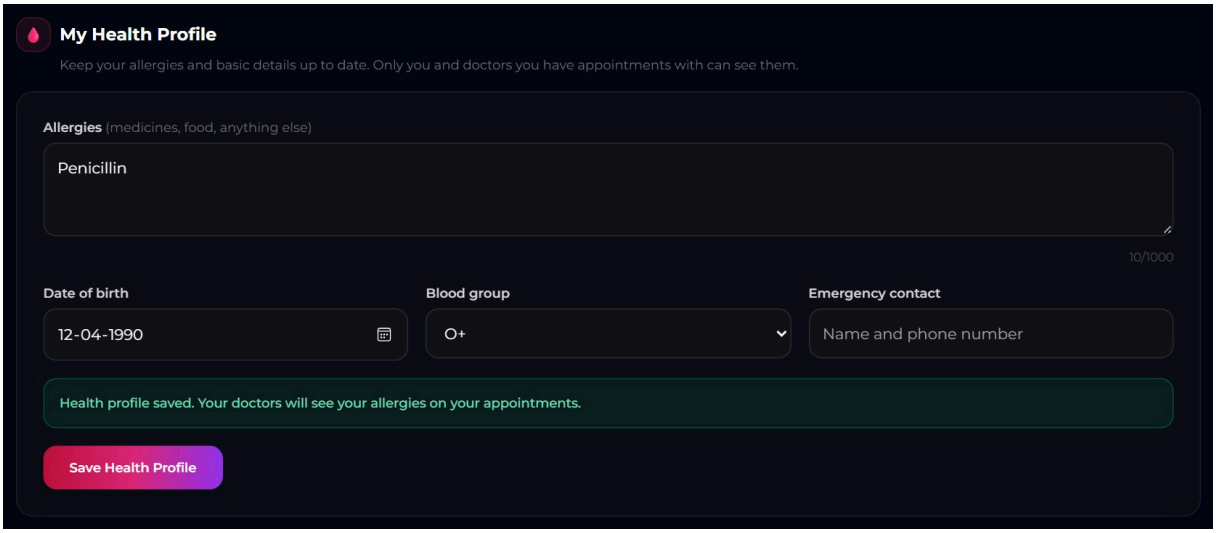


Fig. 5.5 Patient saving their health profile

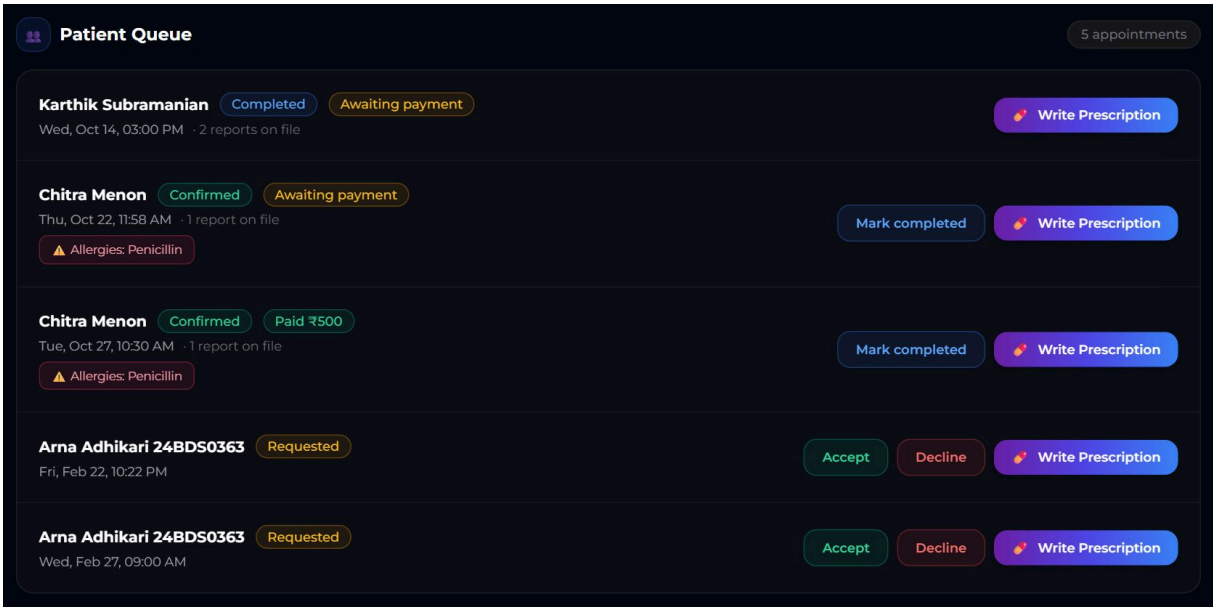


Fig. 5.6 Doctor's appointment queue with accept, decline and complete options

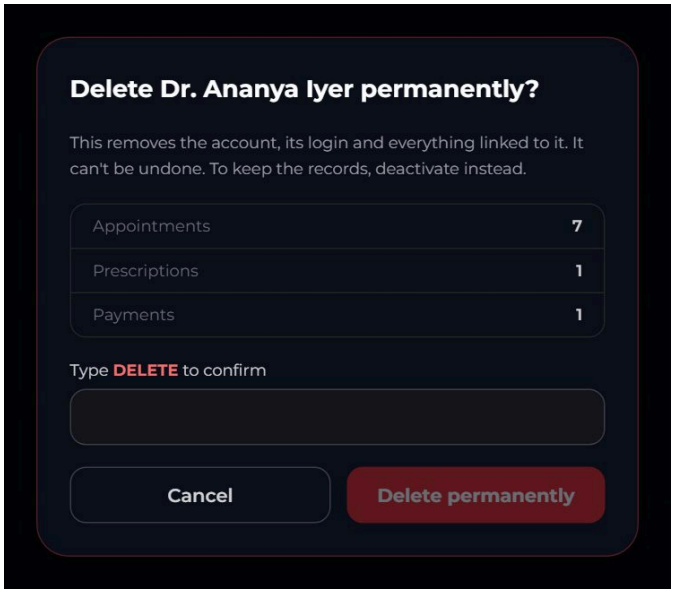


Fig. 5.7 Admin deleting a user after confirmation

5.4 Search

The admin can search users by name or email. The search is done on the server, so it works on all the users and not only the ones on the current page.

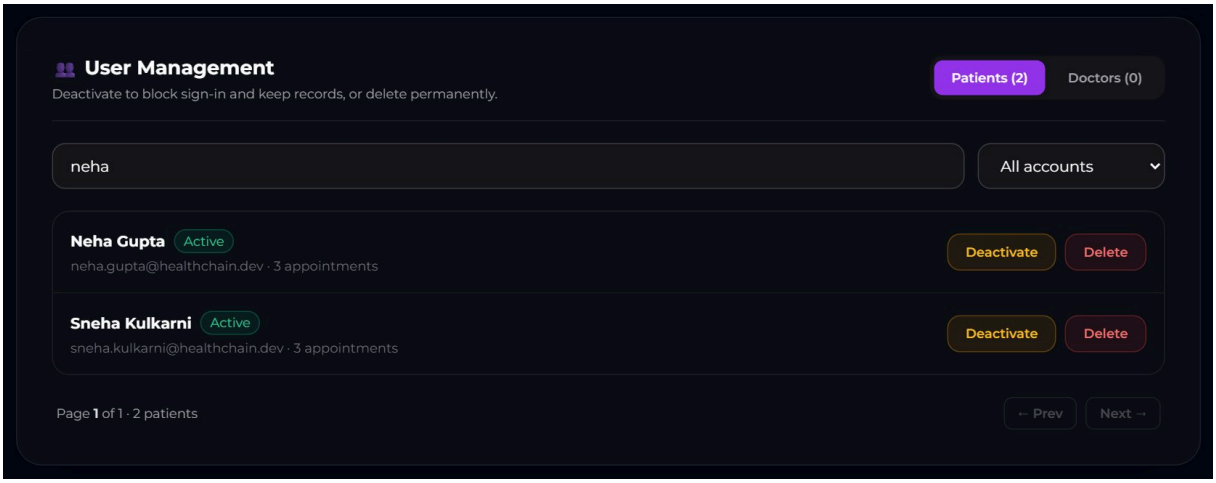


Fig. 5.8 Searching users by name or email

5.5 Filtering

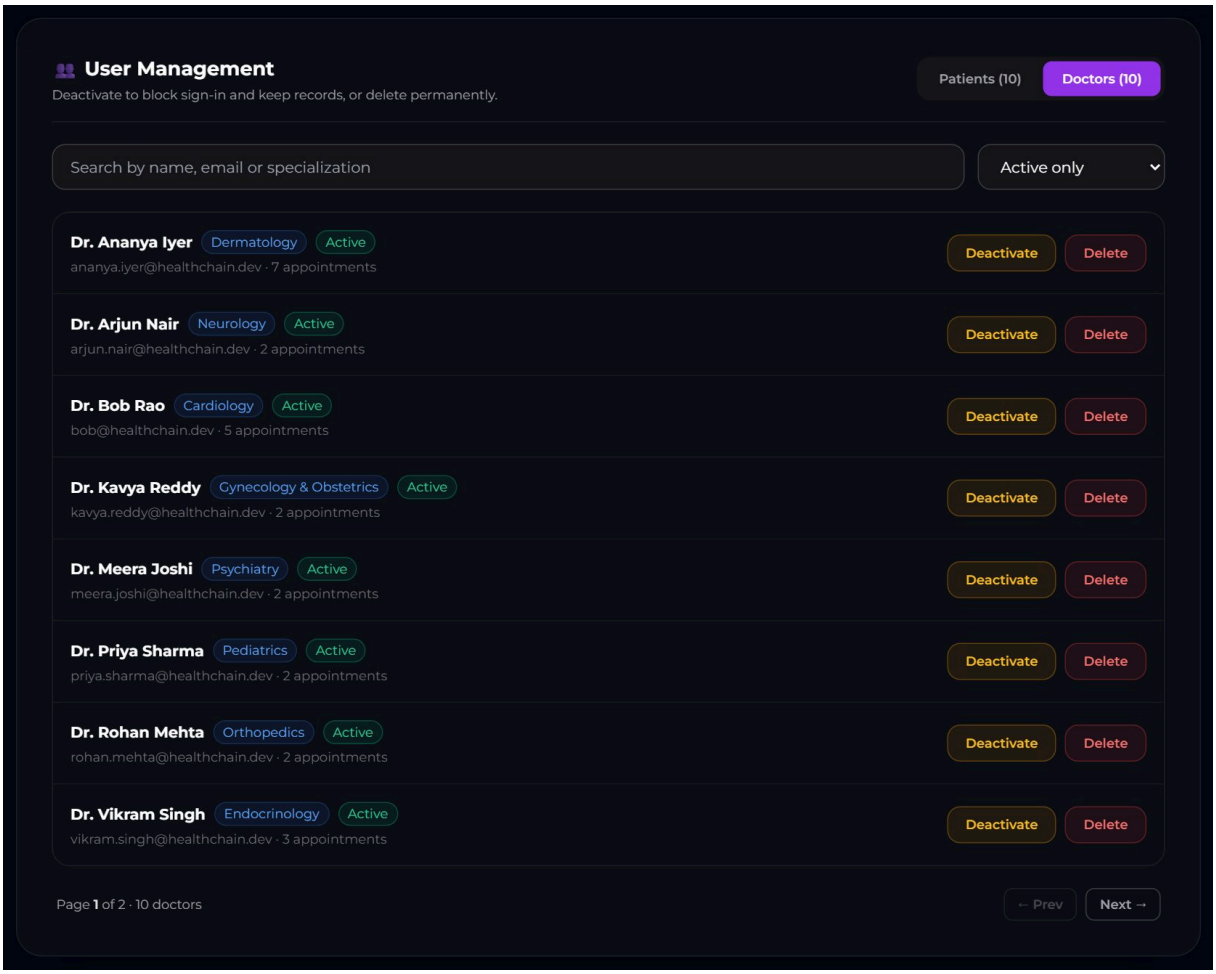


Fig. 5.9 Doctors filtered to show only active accounts

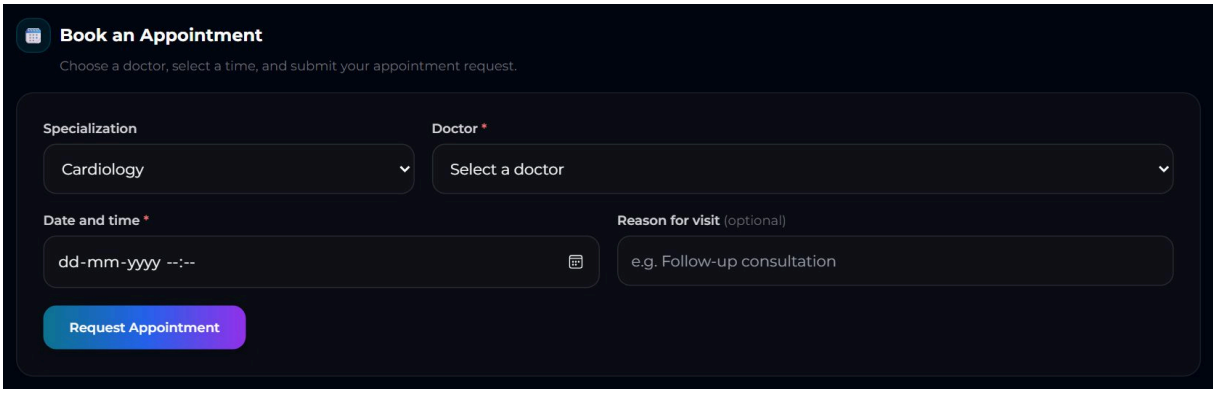


Fig. 5.10 Patient filtering doctors by specialization

5.6 Pagination

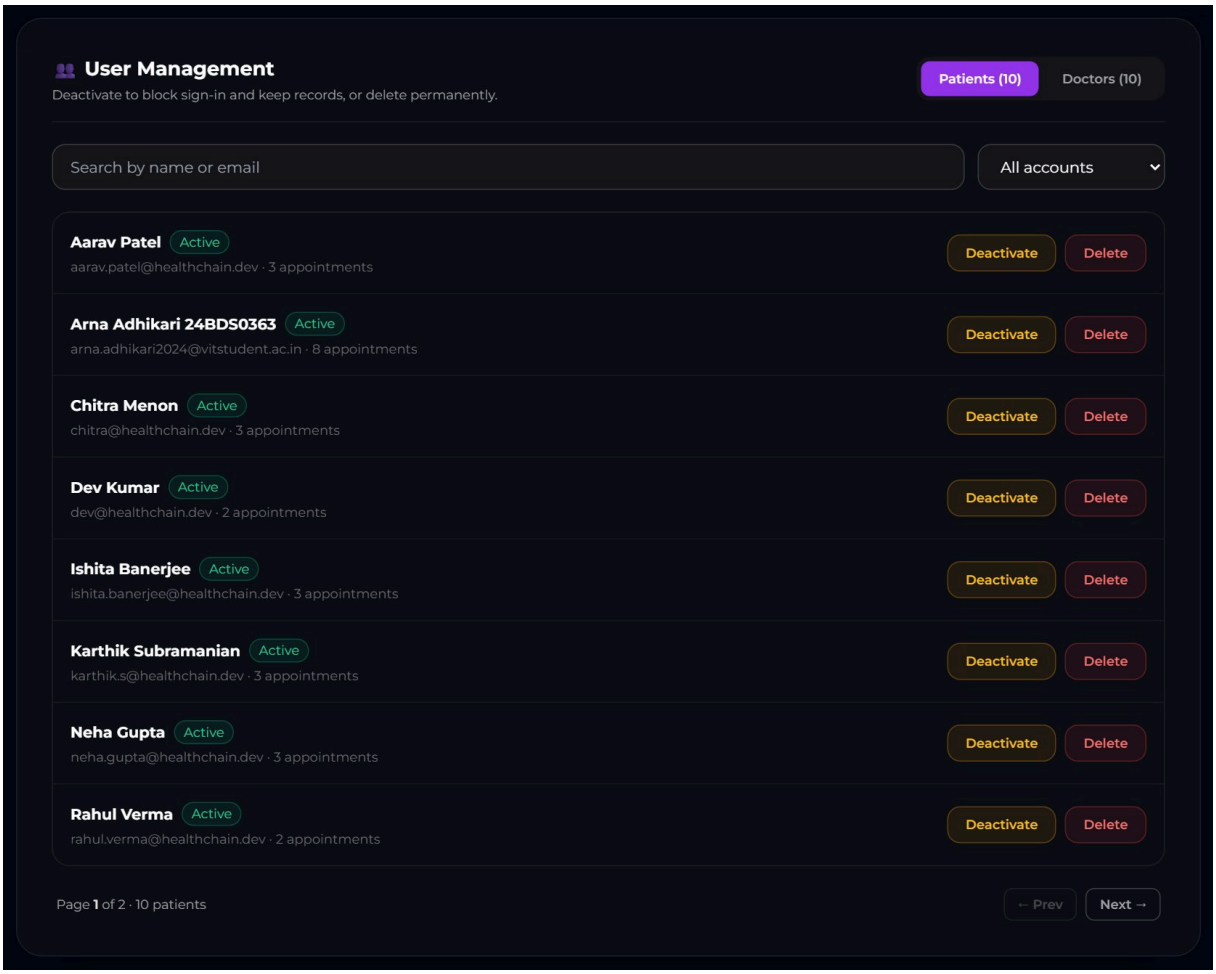


Fig. 5.11 User list, page 1

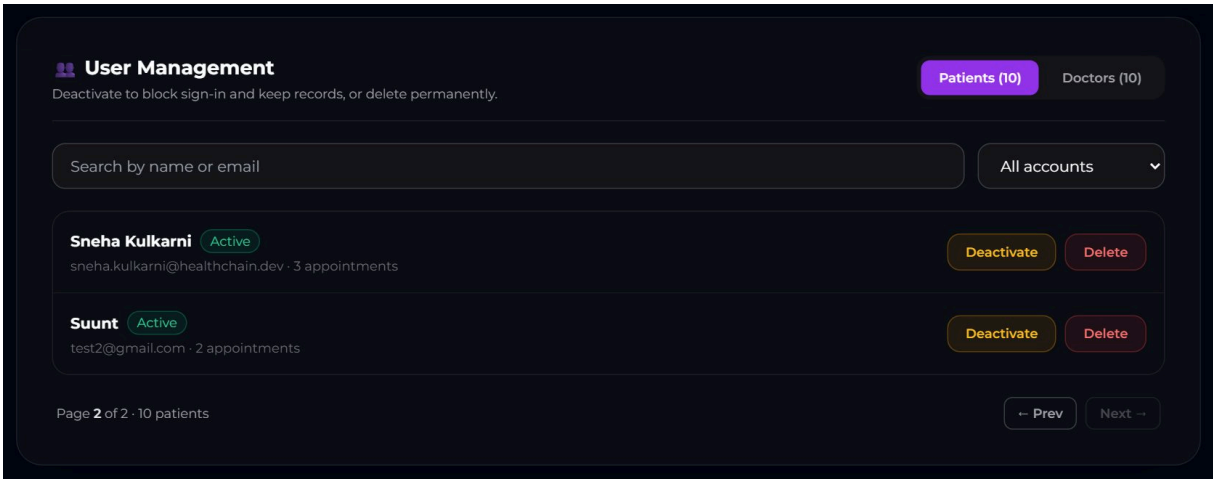


Fig. 5.12 User list, page 2

5.7 Form Validation

Forms are checked in the browser and then again on the server using Zod. So even if someone calls the API directly, invalid data will be rejected. Example:

Request:

```
PUT /api/patients/me/profile

{
  "blood_group": "Z+",
  "date_of_birth": "2099-01-01"
}
```

Response:

```
Status: 400
{
  "error": {
    "formErrors": [],
    "fieldErrors": {
      "date_of_birth": [
        "Date of birth must be in the past."
      ],
      "blood_group": [
        "Invalid enum value. Expected 'A+' | 'A-' | 'B+' | 'B-' | 'AB+' | 'AB-' |
'O+' | 'O-', received 'Z+'"
      ]
    }
  }
}
```

5.8 Exception Handling

All errors on the server go through one error handler, which returns a proper message and status code instead of crashing. It returns 400 for invalid input, 401 if the user is not logged in, 403 for the wrong role, 404 when something does not exist, 409 for duplicates and 503 if the database cannot be reached. Some examples:

Request:

```
GET /api/prescriptions
```

Response:

```
Status: 401
{
  "error": "Missing bearer token"
}
```

Request:

```
GET /api/reports/00000000-0000-0000-0000-000000000000
```

Response:

```
Status: 404
{
  "error": "Not found or not authorized"
}
```

Request:

```
POST /api/auth/register

{
  "email": "chitra@healthchain.dev",
  "password": "••••••••••",
  "full_name": "Duplicate",
  "role": "patient"
}
```

Response:

```
Status: 409
{
  "error": "An account with this email already exists. Please log in."
}
```

5.9 Responsive Design

The application also works on mobile screens. Fig. 5.13 shows the patient dashboard on a phone sized screen.



Fig. 5.13 Patient dashboard on a phone sized screen

6. Authentication and Role-Based Authorization

6.1 Authentication

We have used JWT for authentication. When a user logs in with their email and password, the password is checked by Supabase Auth and our backend gives back a signed token that expires after 12 hours. The token is sent with every request in the Authorization header. For every request, the server checks the token and also checks in the database that the account is still active. So if the admin deactivates someone, they get logged out immediately. Logging out deletes the token from the browser.

Request:

```
POST /api/auth/login

{
  "email": "crud.demo.1793288827949@example.com",
  "password": "••••••••••"
}
```

Response:

```
Status: 200
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5…(JWT)",
  "user": {
    "id": "eb0c8807-df9f-46a9-b299-b54e62b6afc5",
    "role": "patient",
    "full_name": "CRUD Demo Patient",
    "email": "crud.demo.1793288827949@example.com",
    "wallet_address": ""
  }
}
```

The details of the logged in user can be fetched from `GET /api/auth/me` (shown in the CRUD section). After the account is deactivated, the same token does not work any more:

Request:

```
GET /api/auth/me
```

Response:

```
Status: 401
{
  "error": "This account has been deactivated or removed."
}
```

6.2 Role Based Access

There are three roles: patient, doctor and admin. All the users are stored in one `users` table and the `role` column decides what they can do. We did not make a separate table for each role.

Query:

```
SELECT role::text as role, count(*) as users, count(*) filter (where is_active) as
active
FROM users
GROUP BY role
ORDER BY role;
```

Output:

```
 role    | users | active
---------+-------+--------
 admin   |     1 |      1
 doctor  |    10 |     10
 patient |    10 |     10
(3 rows)
```

Access is restricted at three levels:

- **Menus:** every role only sees its own dashboard and the actions related to it.
- **Pages:** before showing a dashboard, the frontend asks the server who the user is. If the role does not match, the user is sent to their own dashboard. To test this, we logged in as

a patient and changed the role saved in the browser to "admin". The admin page still redirected us back to the patient page.

- **API:** protected routes check the role and return 403 if it is wrong. Queries are also limited to the user's own data.

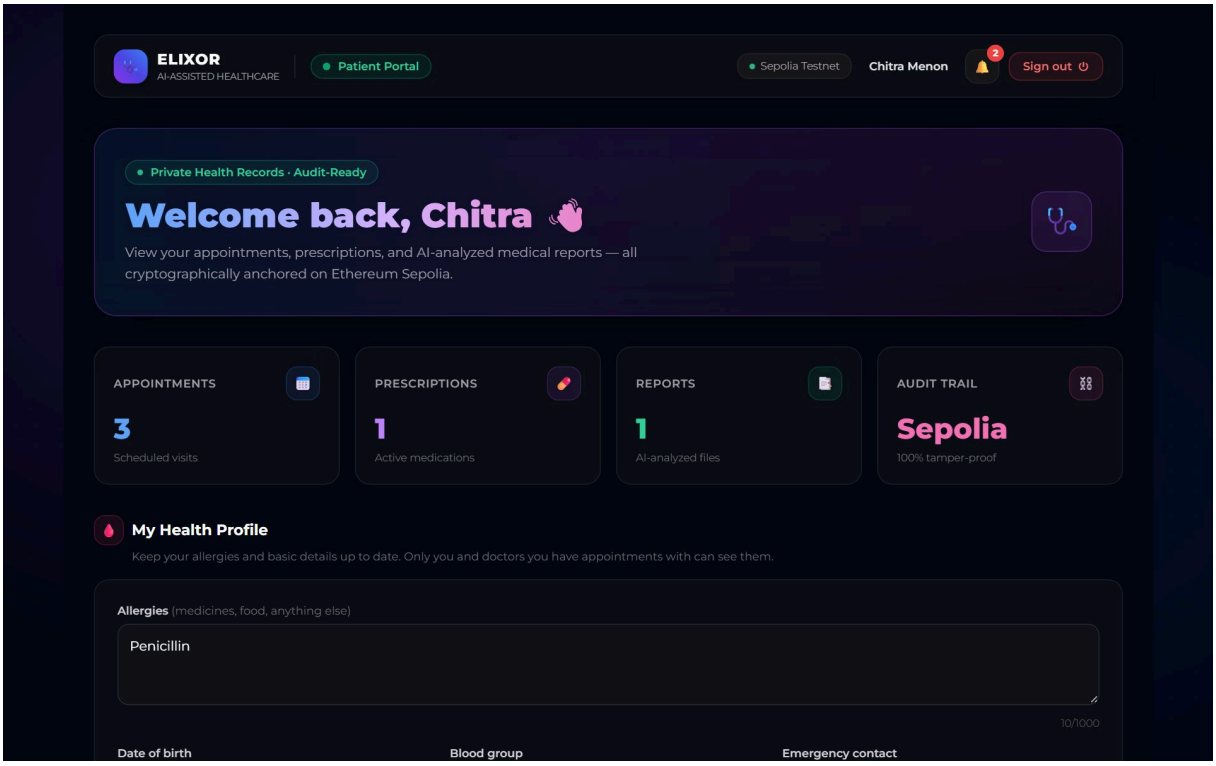


Fig. 6.1 Patient trying to open /admin is sent back to the patient dashboard

Request:

```
GET /api/admin/users?role=patient
```

Response:

```
Status: 403
{
  "error": "Insufficient role"
}
```

7. Database Security

We took the following steps to keep the database secure:

- **Password hashing:** passwords are stored by Supabase Auth as bcrypt hashes. Our own tables do not store any passwords.
- **SQL injection:** all queries go through Prisma, which sends values as parameters. Where we wrote SQL ourselves, we used Prisma's tagged templates which also use parameters. So user input is never directly added into an SQL string.

- **Secrets:** the database URL, JWT secret and API keys are kept in `backend/.env` and loaded using dotenv. The .env files are in .gitignore and are not pushed to GitHub. Only an example file with dummy values is in the repository.
- **Row level security:** as an extra layer, RLS is turned on for every table, with policies that only allow users to access their own rows.

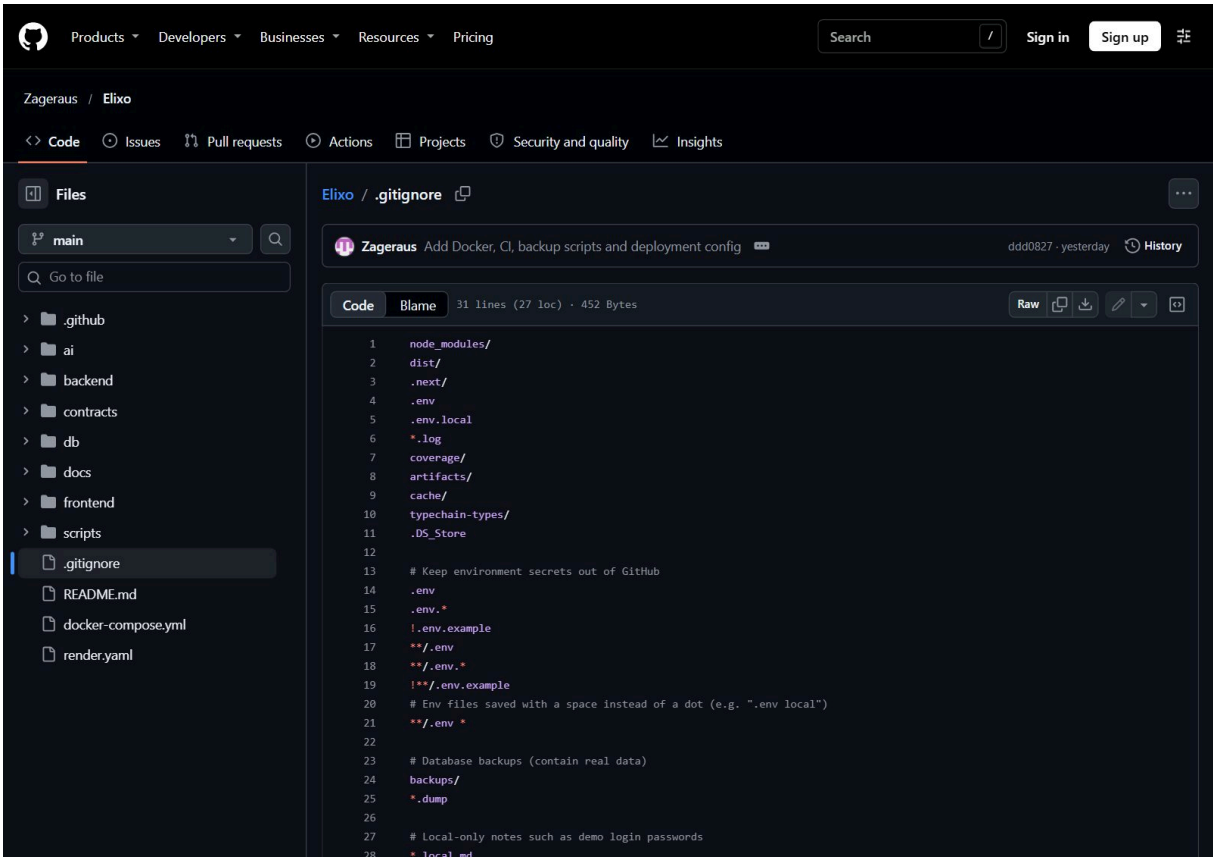


Fig. 7.1 Our .gitignore file, which excludes environment files and backups

Query:

```
SELECT tablename as table_name, policyname as policy, cmd as command
FROM pg_policies
WHERE schemaname = 'public'
ORDER BY 1, 2;
```

Output:

```
 table_name               | policy                         | command
--------------------------+--------------------------------+---------
 ai_analyses              | p_ai_analyses_select           | SELECT
 appointments             | p_appointments_insert          | INSERT
 appointments             | p_appointments_select          | SELECT
 appointments             | p_appointments_update          | UPDATE
 audit_logs               | p_audit_admin_only             | SELECT
 blockchain_transactions | p_btx_select                    | SELECT
 doctor_profiles          | p_doctor_profiles              | ALL
 medical_reports          | p_reports_insert               | INSERT
 medical_reports          | p_reports_select               | SELECT
 notifications            | p_notifications_owner          | ALL
 patient_profiles         | p_patient_profiles             | ALL
 payments                 | p_payments_select              | SELECT
 prescription_items       | p_prescription_items_select | SELECT
 prescriptions            | p_prescriptions_insert         | INSERT
 prescriptions            | p_prescriptions_select         | SELECT
 users                    | p_users_select_for_booking     | SELECT
 users                    | p_users_self_update            | UPDATE
(17 rows)
```

8. Database Backup and Recovery

For backups we wrote two scripts that use PostgreSQL's own tools. `scripts/db-backup.sh` runs pg_dump in custom format and saves a file such as `backups/elixor_20261030_1015.dump`. `scripts/db-restore.sh` restores this file using pg_restore. The restore runs as a single transaction, so if it fails in the middle, nothing is half restored. The backups folder is added to .gitignore because it has real data.

Commands used:

```
bash scripts/db-backup.sh
pg_restore --list backups/elixor_<timestamp>.dump
RESTORE_URL=<test database> bash scripts/db-restore.sh
backups/elixor_<timestamp>.dump
```

[Insert screenshot: terminal output of the backup script]

[Insert screenshot: terminal output of the restore into a test database]

9. Version Control

Our final project is on GitHub at https://github.com/Zageraus/Elixo and the repository is public. We use two branches, `main` and `dev`. New work is done on dev and then merged into main through a pull request (PR #1 "Dev", merged on 28 October 2026). The README explains what the project is, the tech stack, and how to install and run it. The .gitignore file keeps secrets and build files out of the repository.

9.1 Repositories Used During the Project

The project moved through a few repositories before we settled on the final one. Each later repository carries the history of the one before it, so all the commits can still be seen in the final repository.

Repository	Created	What it was used for
https://github.com/Zageraus/Healthcare	22 Sep 2026	First repository, created when we picked the topic. No code was pushed to it.
https://github.com/Arnahhh/HealthChain-Finalised	16 Oct 2026	Arna's upload of the whole project, including the AI service and smart contract (1 commit).
https://github.com/Arnahhh/HealthChain-AI-and-BlockChain-integrated-	19 Oct 2026	The project after the AI and blockchain upgrade was merged (PR #1, healthchain-upgrade). 10 commits.
https://github.com/Arnahhh/Elixor-update	20 Oct 2026	The project after it was renamed to ELIXOR. 12 commits.
https://github.com/Zageraus/Elixo	25 Oct 2026	Final repository with the database work, improvements and deployment. Contains the full history (42 commits).

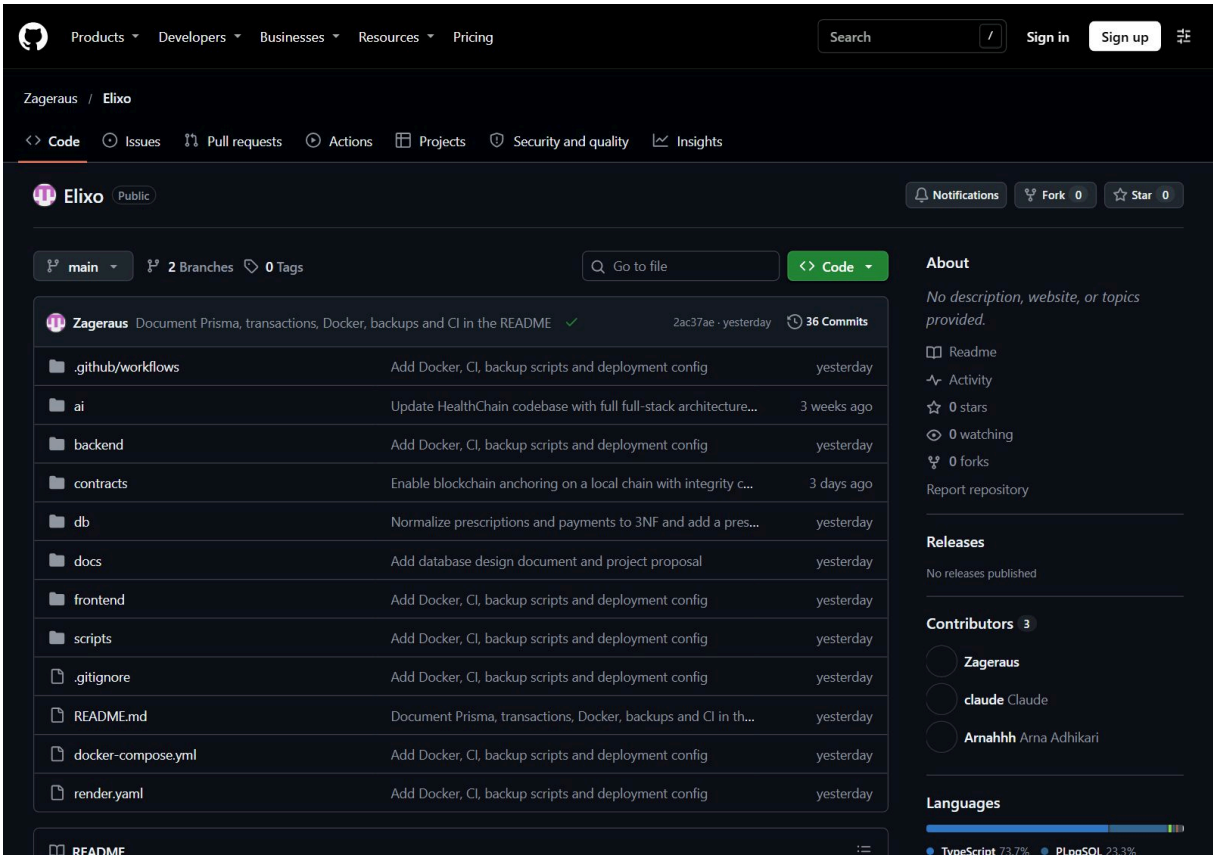


Fig. 9.1 Our GitHub repository

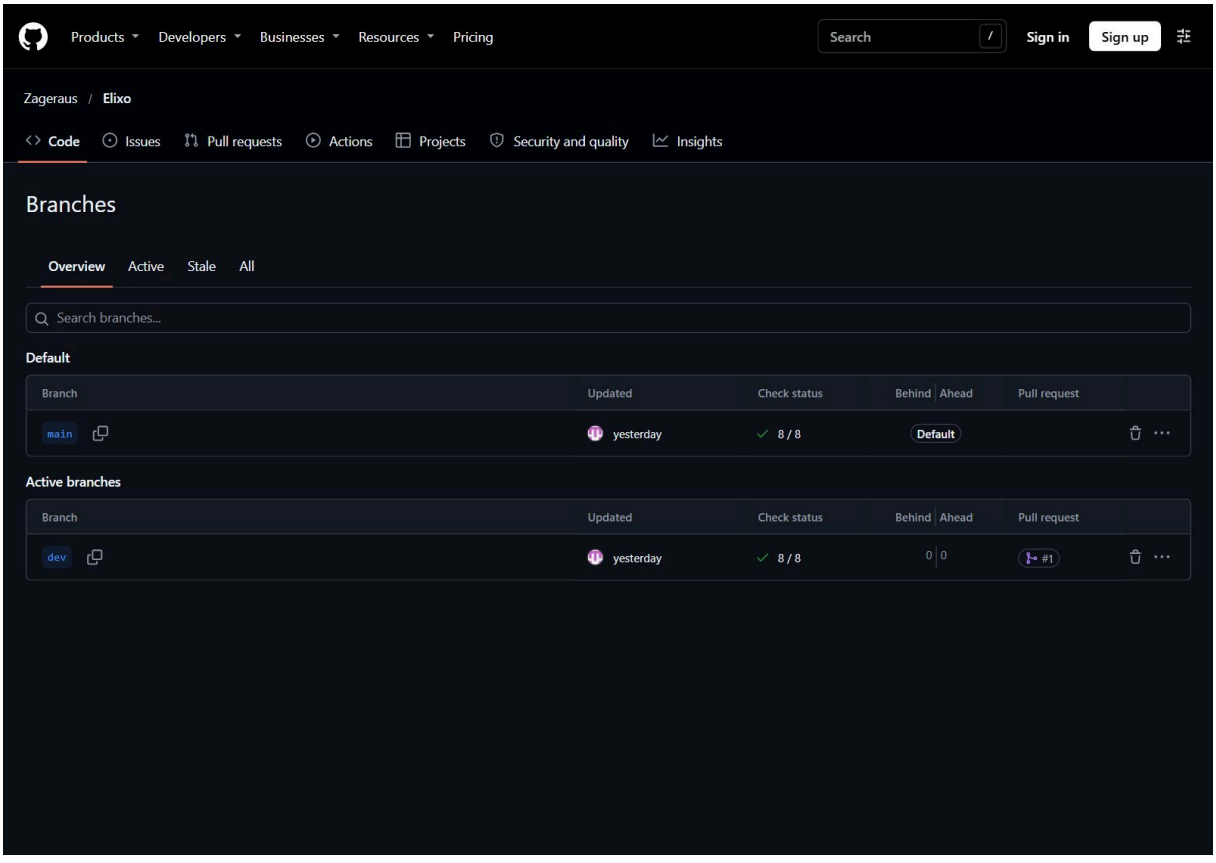


Fig. 9.2 Branches main and dev

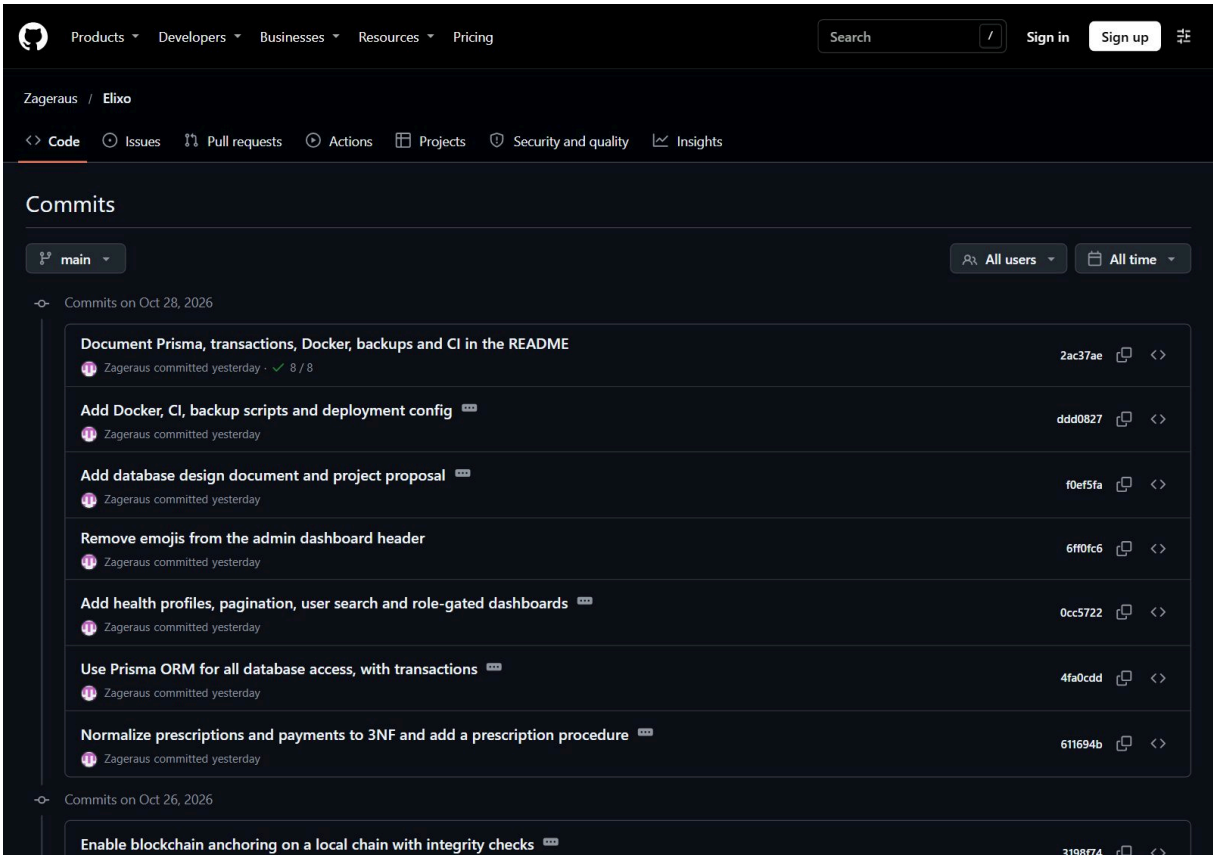


Fig. 9.3 Commit history

10. Deployment

The application is deployed and can be used online. The three parts are hosted separately:

Part	Hosted on	URL
Frontend (Next.js)	Vercel	https://elixor-dun.vercel.app
Backend API (Express.js)	Render	https://elixor-backend.onrender.com
Database (PostgreSQL 17)	Supabase	Managed, not public

The frontend gets the address of the API from the environment variable `NEXT_PUBLIC_API_URL`, which we set in Vercel. On the backend, the database URL, JWT secret and other keys are set as environment variables in Render, so they are not in the code. The backend only accepts requests from our Vercel domain (CORS). Our CI pipeline also builds the Docker images of both parts on every push, and `render.yaml` and the Dockerfiles are in the repository.

Two features needed a different setup on the live server than on our laptops:

- **AI summaries:** locally the backend uses an Ollama model, but Render has no Ollama. So on Render we set `AI_PROVIDER=openai-compatible` and a free Groq API key (`AI_API_KEY`), and the summaries are written by a model on Groq's free API (`openai/gpt-oss-120b` by default). The backend asks for the answer in JSON and checks it before saving it to `ai_analyses`. If the model is not available for our key, the backend asks Groq for the list of models and picks another one.
- **Blockchain anchoring:** anchoring needs a running blockchain node, which we only run locally (Hardhat). On Render we set `BLOCKCHAIN_ENABLED=false`, so the live site skips anchoring and says so in the upload message. Anchoring and the integrity check work when the project is run locally with the Hardhat chain.

Live URL: https://elixor-dun.vercel.app

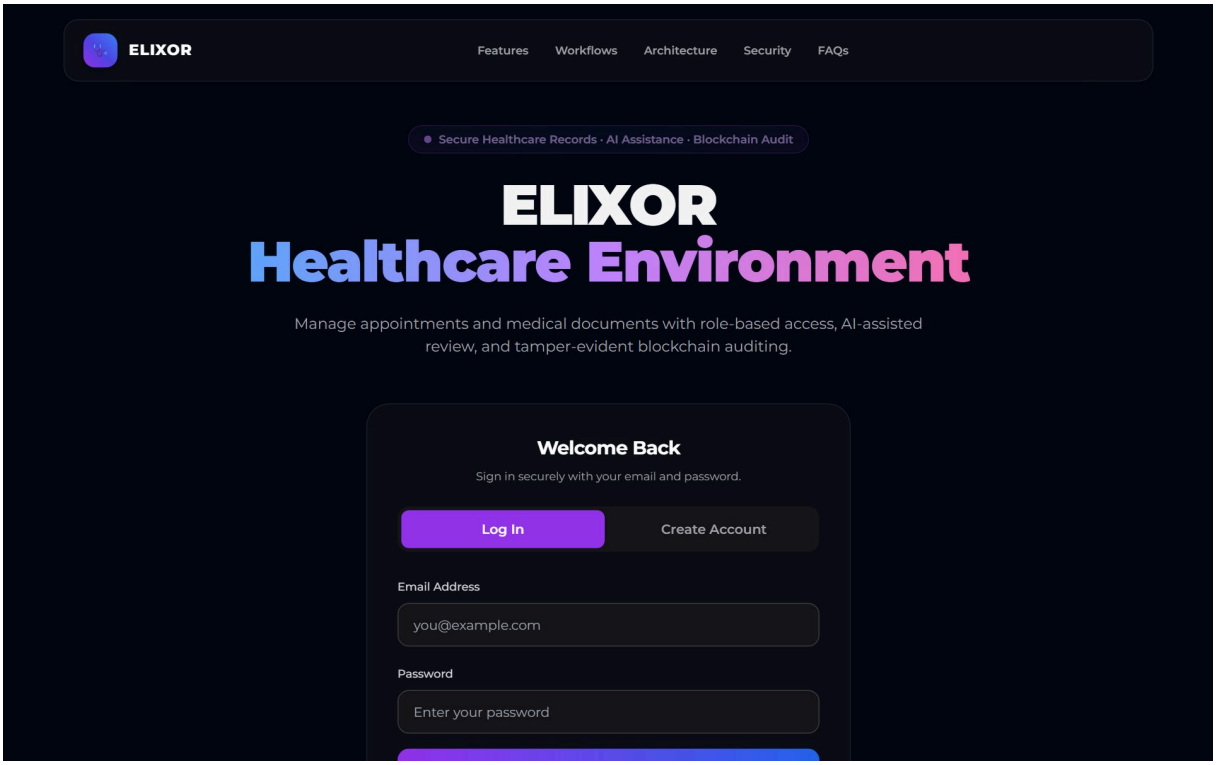


Fig. 10.1 ELIXOR running live at https://elixor-dun.vercel.app

To check that the deployed backend is running and connected to the database, we called its health endpoint:

Request:

```
GET https://elixor-backend.onrender.com/health
```

Response:

```
Status: 200
{
  "status": "ok",
  "service": "healthchain-backend",
  "database": "ok"
}
```

We also checked that the API allows requests from the frontend's domain. A CORS preflight request from https://elixor-dun.vercel.app returned `204 No Content` with `Access-Control-Allow-Origin: https://elixor-dun.vercel.app`.

11. Bonus

11.1 CI/CD using GitHub Actions

Whenever we push to main or dev, our workflow runs automatically. It validates the Prisma schema, type checks and builds the backend and frontend, runs the smart contract tests and builds the Docker images.

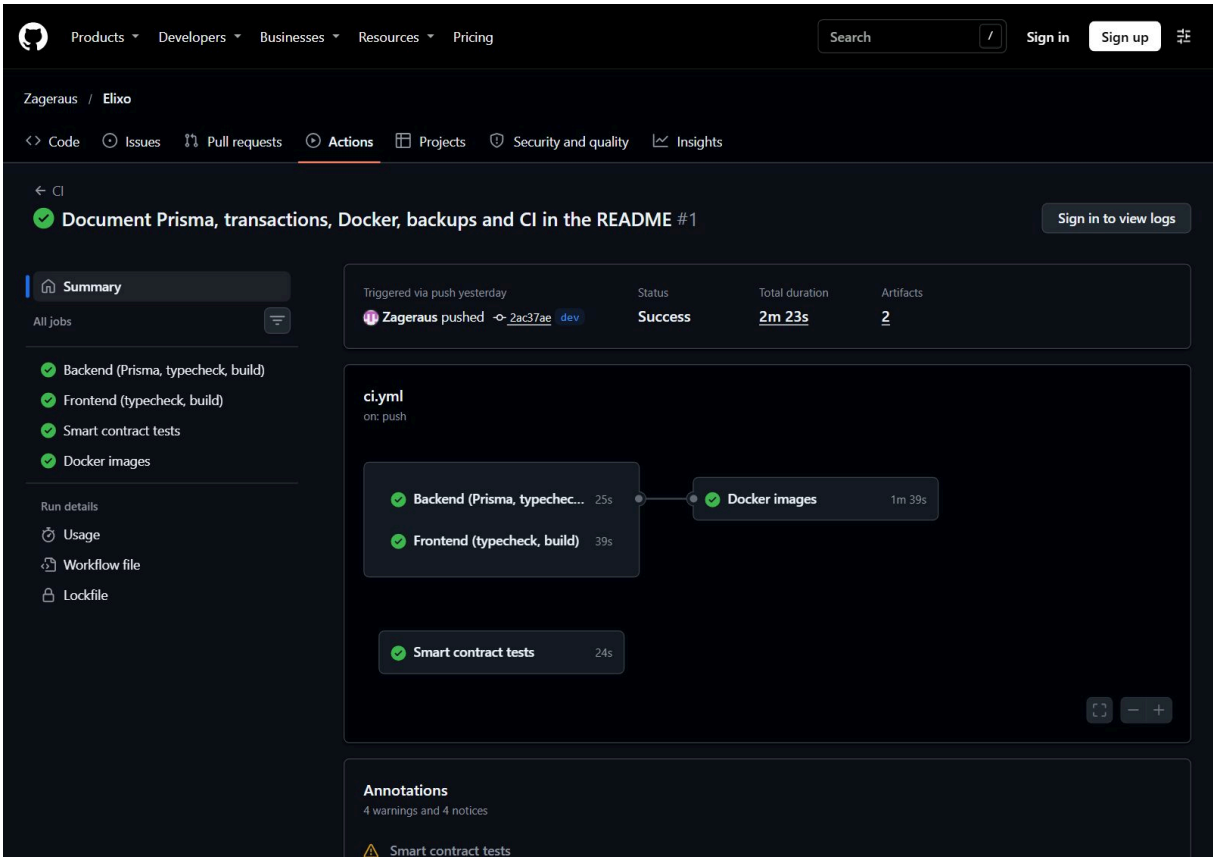


Fig. 11.1 A successful CI run

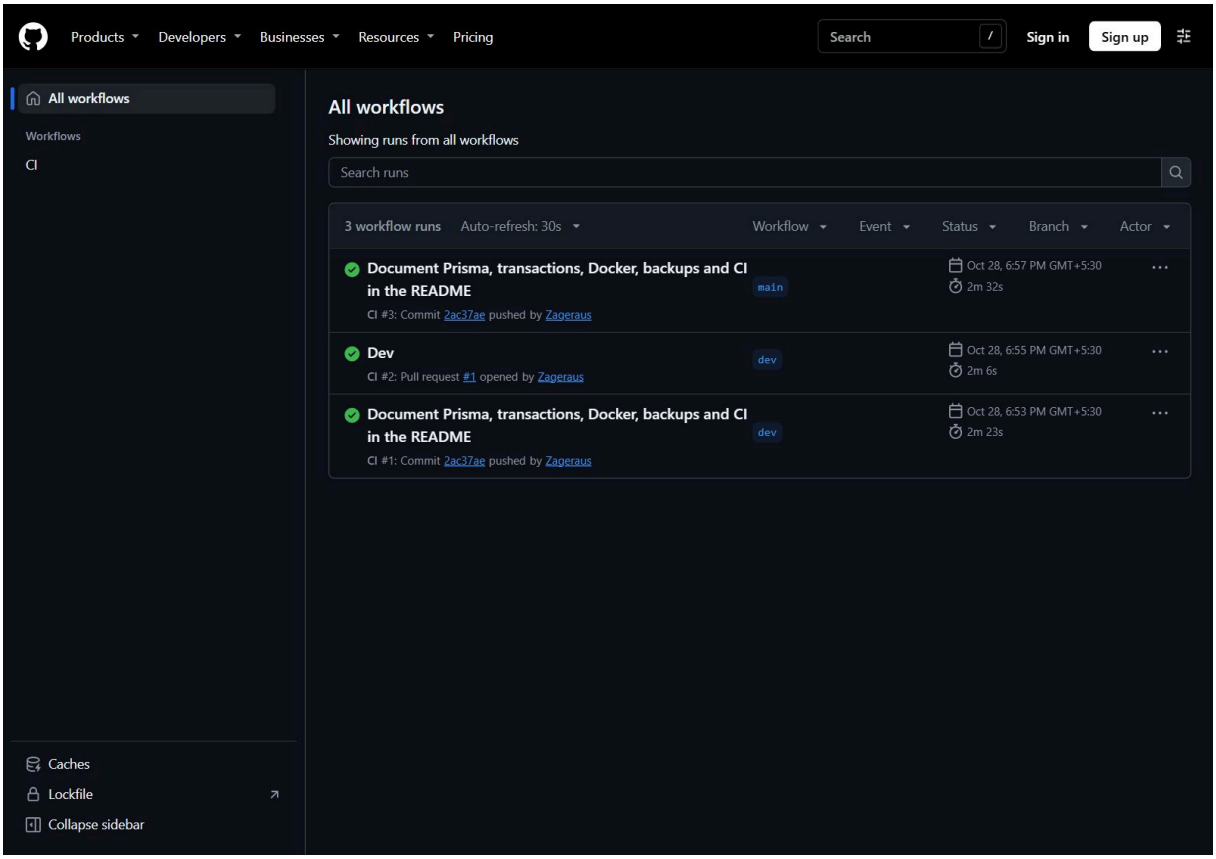


Fig. 11.2 List of CI runs

11.2 Other Practices

We used a managed cloud database (Supabase), an audit log that is filled automatically by triggers and also stores which user made the change, automated tests in CI, and indexes based on the queries our app actually runs. The query below shows the latest entries in the audit log:

Query:

```
SELECT to_char(l.created_at at time zone 'Asia/Kolkata', 'DD Mon HH24:MI') as at_ist,
coalesce(u.full_name, '(system)') as acted_by, l.action, l.entity_type
FROM audit_logs l
  LEFT JOIN users u on u.id = l.user_id
ORDER BY l.created_at desc
LIMIT 10;
```

Output:

```
 at_ist       | acted_by     | action | entity_type
--------------+--------------+--------+--------------
 28 Oct 18:31 | (system)     | DELETE | payments
 28 Oct 18:31 | (system)     | DELETE | appointments
 28 Oct 18:31 | (system)     | DELETE | prescriptions
 28 Oct 18:29 | Dr. Bob Rao  | UPDATE | payments
 28 Oct 18:29 | Dr. Bob Rao  | UPDATE | appointments
 28 Oct 18:29 | Chitra Menon | INSERT | payments
 28 Oct 18:29 | Dr. Bob Rao  | INSERT | prescriptions
 28 Oct 18:29 | Dr. Bob Rao  | UPDATE | appointments
 28 Oct 18:29 | Chitra Menon | INSERT | appointments
 28 Oct 16:09 | (system)     | INSERT | prescriptions
(10 rows)
```

12. Project Execution

In this section we follow one patient visit through the whole system. For every step we have written the input, what happens in the system (processing), and the output.

12.1 Patient Books an Appointment

Input: the patient selects a specialization, a doctor, a date and a reason.
Processing: the API checks the token and role, validates the data and inserts the appointment inside a transaction. A trigger notifies the doctor and writes an audit record.
Output: the appointment is shown with status "requested".

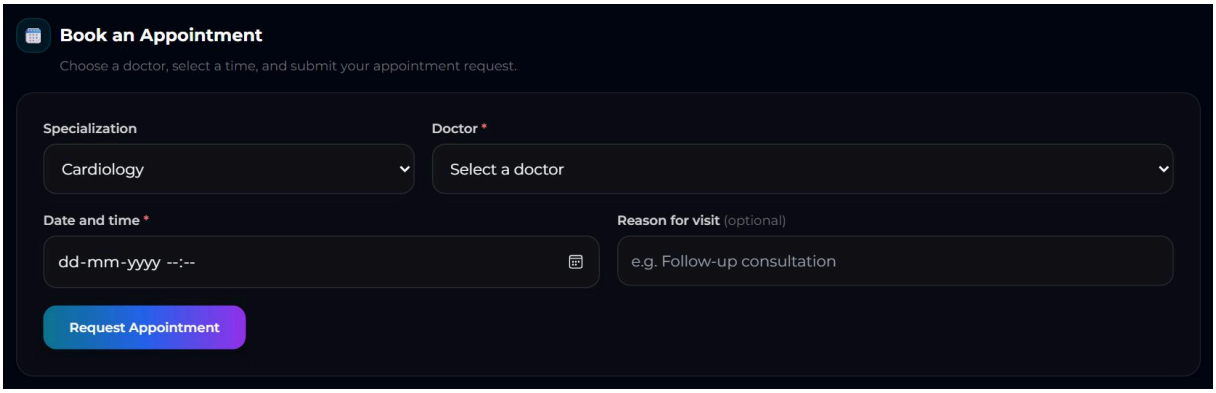


Fig. 12.1 Booking form

12.2 Doctor Sees the Request

Input: the doctor opens their dashboard.
Processing: the API reads the doctor's appointments from the `v_doctor_dashboard` view and adds the payment status and the patient's allergies.
Output: the appointment queue, with the patient's allergies highlighted.

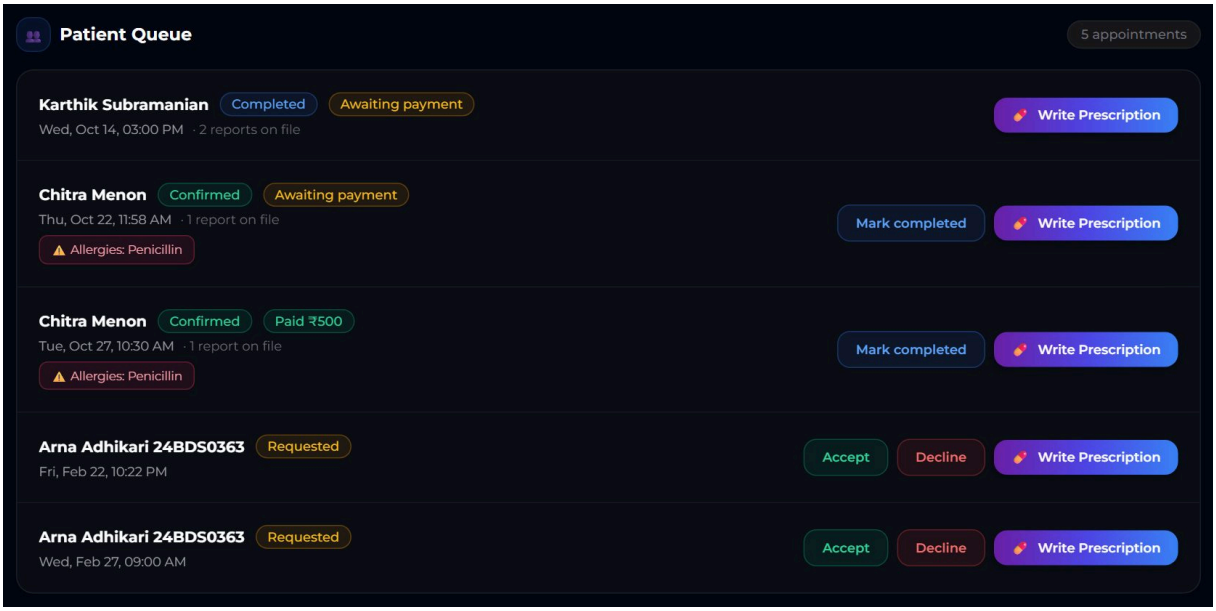


Fig. 12.2 Doctor's appointment queue

12.3 Doctor Writes a Prescription

Input: patient, medicines and notes.

Processing: the API calls the stored procedure inside a transaction. Triggers check consistency, write the audit log and notify the patient.

Output: the prescription is saved and shown to both the doctor and the patient.

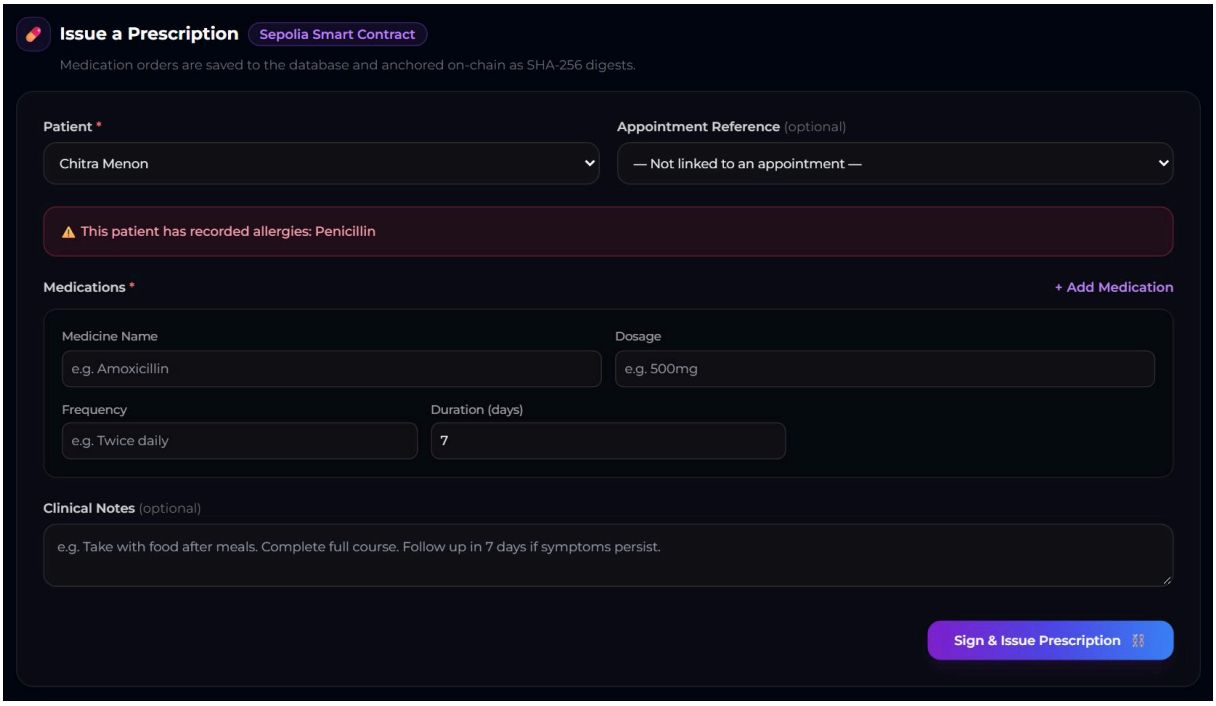


Fig. 12.3 Prescription form with the allergy warning

12.4 Patient Views the Prescription and Pays

Input: payment method.
Processing: the payment transaction locks the appointment, checks that it is confirmed and not already paid, and saves the payment using the doctor's fee from the database.
Output: a receipt is generated and the appointment is shown as paid.

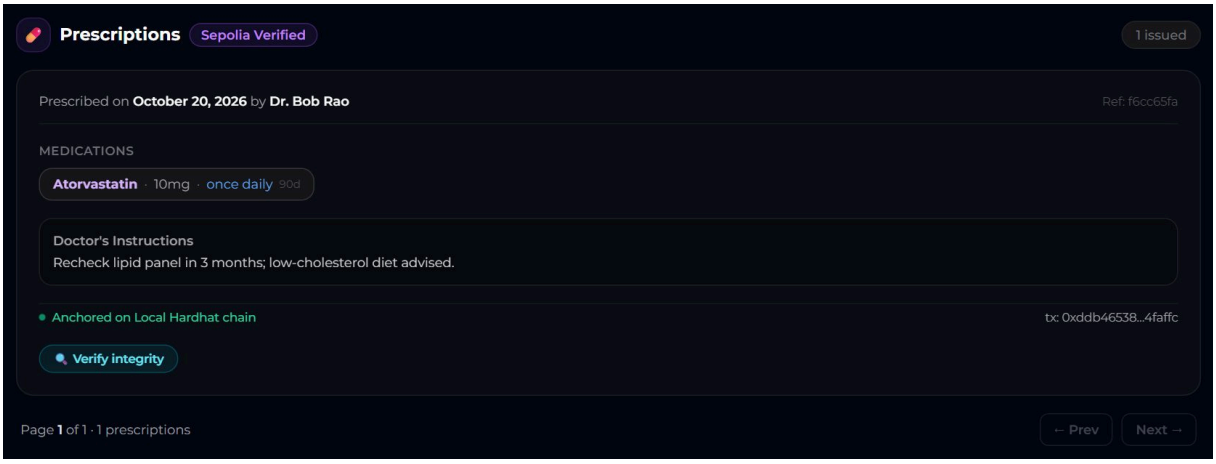


Fig. 12.4 Patient's prescriptions

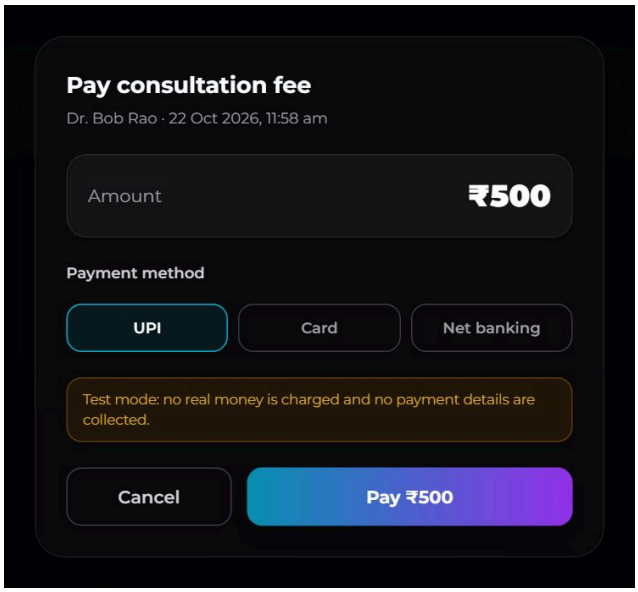


Fig. 12.5 Checkout page (simulated, no real payment)

12.5 Patient Uploads a Report

Input: title, type, file and optionally the text of the report.
Processing: the file is stored privately, its SHA-256 hash is saved, and the text is summarised by the AI model (Groq's free API on the live server, Ollama when we run it locally).
Output: the report is shown along with its AI summary.

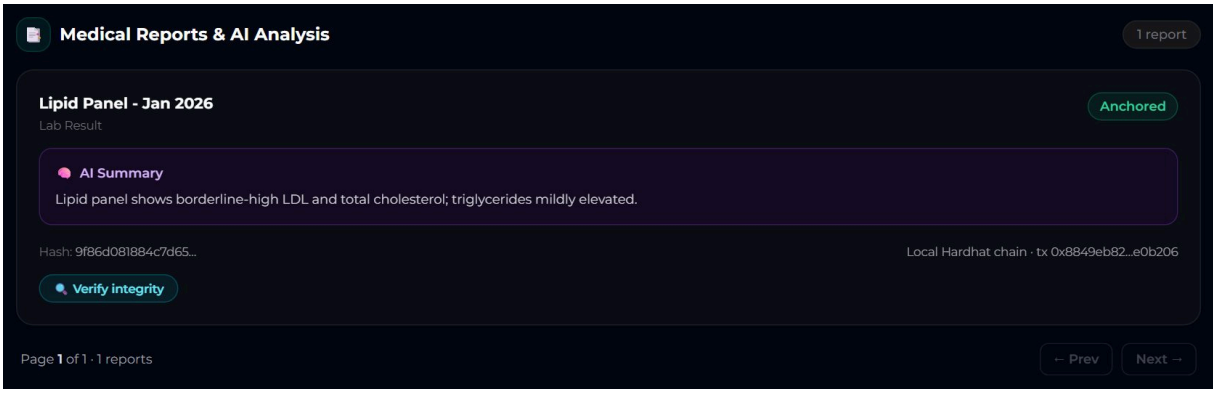


Fig. 12.6 Reports with AI summaries

12.6 Admin Checks the Platform

Input: the admin opens the dashboard.
Processing: the API reads from `v_admin_analytics` and `v_doctor_weekly_load`.
Output: totals, revenue, refunds and a weekly load chart, along with user management.

Query:

```
SELECT *
FROM v_admin_analytics;
```

Output:

```
 total_patients | total_doctors | total_appointments | completed_appointments |
total_reports | total_ai_analyses | total_prescriptions | confirmed_chain_txs |
pending_chain_txs | avg_confirmation_seconds | total_revenue | paid_payments |
total_refunded | awaiting_payment
----------------+---------------+--------------------+------------------------+------
---------+-------------------+---------------------+---------------------+-----------
--------+---------------------------+---------------+---------------+----------------
+------------------
              10 |            10 |                 32 |                      9 |
11 |                 9 |                  10 |                  21 |
0 |                     0.3 |         10500 |            11 |               0 |
6
(1 row)
```

13. Improvements

At our first review, the project only had the login page and appointment booking. This was the state of the code up to commit a966cbe ("Add email auth, appointment booking, and optional blockchain anchoring", 25 October 2026). Everything listed below was added after the review. Any commit can be opened at https://github.com/Zageraus/Elixo/commit/<ID>.

Commit ID	Date	Commit message	What we changed

`7a51624`	25 Oct 2026 15:29	Guard patient dashboard by role and surface doctor-list errors	The patient dashboard now checks the user's role, and an error while loading the doctor list is shown instead of an empty list.
`9390d47`	25 Oct 2026 15:31	Sign out automatically when the stored session is rejected	If the saved login is no longer valid, the user is signed out automatically.
`518f977`	25 Oct 2026 15:33	Fix form reset after booking and report upload	Fixed the booking and report upload forms not clearing after they were submitted.
`a493dfa`	25 Oct 2026 15:43	Let doctors accept appointments and scope dashboard data to the user	Doctors can accept or decline appointments, and each dashboard only shows the logged in user's own data.
`ae28df1`	25 Oct 2026 15:55	Collect doctor specialization at sign-up and filter doctors by it	Doctors choose their specialization when signing up, and patients can filter doctors by it while booking.
`e6f2c21`	25 Oct 2026 16:11	Add demo seed data and enable RLS on auth_nonces	Added demo seed data (doctors, patients, appointments, prescriptions and reports) and turned on RLS for auth_nonces.
`3050504`	25 Oct 2026 16:29	Show names on prescriptions and only badge truly anchored ones	Prescriptions show the doctor's and patient's names, and only prescriptions that are really anchored on the blockchain get the anchored badge.
`c3a2945`	25 Oct 2026 16:41	Add simulated consultation payments	Added simulated consultation payments: payments table, checkout page and receipt.
`d4ba353`	25 Oct 2026 16:48	Let admins deactivate or permanently delete patients and doctors	Admins can deactivate or permanently delete patients and doctors.
`6773bd4`	25 Oct 2026 16:54	Notify patients when their doctor is deactivated or deleted	Patients get a notification when their doctor is deactivated or deleted.
`1835523`	25 Oct 2026 17:03	Fix permanent delete of users with anchored records	Fixed deleting users who have anchored records (migration 010 adds ON DELETE CASCADE to blockchain_transactions).
`c22f815`	26 Oct 2026 20:38	Make the report upload message reflect what actually happened	The upload message now says what actually happened with the AI summary and the anchoring.
`d458724`	26 Oct 2026 20:52	Show the real anchoring status on report cards	Report cards show the real anchoring status of each report.
`4d66d95`	26 Oct 2026 20:59	Let patients open uploaded report PDFs through signed links	Patients can open their uploaded PDFs through short-lived signed links.

`174f6b5`	26 Oct 2026 21:07	Rewrite README to match the current project	Rewrote the README to match the project.
`3198f74`	26 Oct 2026 21:33	Enable blockchain anchoring on a local chain with integrity checks	Blockchain anchoring works on a local Hardhat chain, with an integrity check that compares the hash in the database with the one on the chain.
`611694b`	28 Oct 2026 18:50	Normalize prescriptions and payments to 3NF and add a prescription procedure	Added migration 012. The medicines were moved to a new prescription_items table, the payment columns that depended on the appointment were dropped, and we added the stored procedure, a trigger to keep prescriptions consistent and some new indexes.
`4fa0cdd`	28 Oct 2026 18:50	Use Prisma ORM for all database access, with transactions	Changed all database access to Prisma. Operations with more than one step now run inside transactions, the audit log saves who made each change, and we added /api/auth/me, the health profile API, server side pagination and one common error handler.
`0cc5722`	28 Oct 2026 18:50	Add health profiles, pagination, user search and role-gated dashboards	Frontend changes for the above: health profile form, allergy warning for doctors, page numbers on lists, admin search and filter, and the RoleGate check on every dashboard.
`6ff0fc6`	28 Oct 2026 18:50	Remove emojis from the admin dashboard header	Small change to the admin dashboard header.
`f0ef5fa`	28 Oct 2026 18:50	Add database design document and project proposal	Wrote the database design document (ER diagram, schema, data dictionary and normalization) and the project proposal with the Gantt chart.
`ddd0827`	28 Oct 2026 18:50	Add Docker, CI, backup scripts and deployment config	Added Dockerfiles, docker-compose, the GitHub Actions workflow, the backup and restore scripts and the Render config.
`2ac37ae`	28 Oct 2026 18:50	Document Prisma, transactions, Docker, backups and CI in the README	Updated the README for these changes.
`7bfab2a`	29 Oct 2026 21:19	Return 409 when registering an email that is already in use	Registering with an email that already exists now gives 409 instead of 400. We noticed this while testing for this report.
`18918c0`	29 Oct 2026 21:19	Add a plain-language project overview PDF	Added a short overview document of the project.

3959539	30 Oct 2026 12:21	Listen on 0.0.0.0 for Render and ignore the .vercel folder	The backend now listens on 0.0.0.0 so that Render can reach it, and the .vercel folder is ignored.
6530d61	30 Oct 2026 12:21	Use Claude for report summaries on the deployed server	On the deployed server there is no Ollama, so AI summaries failed. Added Claude (Anthropic API) as a hosted AI option (AI_PROVIDER=claude); Ollama is still used locally.
925e4ec	30 Oct 2026 21:35	Add a free OpenAI-compatible AI provider (Groq) for live summaries	Claude is a paid API, so we added a free option: any OpenAI-compatible chat API, with Groq's free tier as the default (AI_PROVIDER=openai-compatible). This is what the live server uses. A wrong AI_PROVIDER now gives a clear error instead of quietly using Ollama.
b0e7e8b	30 Oct 2026 21:40	Pick an available AI model when the configured one is gone	Groq said the model we had set was not available for our key. Now, if a model is missing, the backend asks Groq which models the key can use and picks one, with openai/gpt-oss-120b as the default.

Compared to the version we showed in the first review, the main changes are:

- Doctors can now accept, decline and complete appointments, and patients can filter doctors by specialization.
- Consultation payments were added, with an automatic refund (trigger) when a paid appointment is cancelled.
- Doctors can write prescriptions, and patients can save a health profile with their allergies, which doctors see as a warning.
- Patients can upload medical reports and get an AI summary of them, and reports and prescriptions can be anchored on the blockchain and checked for tampering.
- The admin can search, filter, page through, deactivate and delete users, and patients are notified when their doctor is removed.
- The schema was normalized further (new prescription_items table, payment columns removed) and we added a stored procedure.
- All database access was moved to the Prisma ORM, and operations with many steps now run inside transactions.
- Every dashboard checks the user's role with the server before it is shown.
- We added the design documents, Docker setup, CI pipeline and backup scripts, and deployed the app on Vercel and Render.

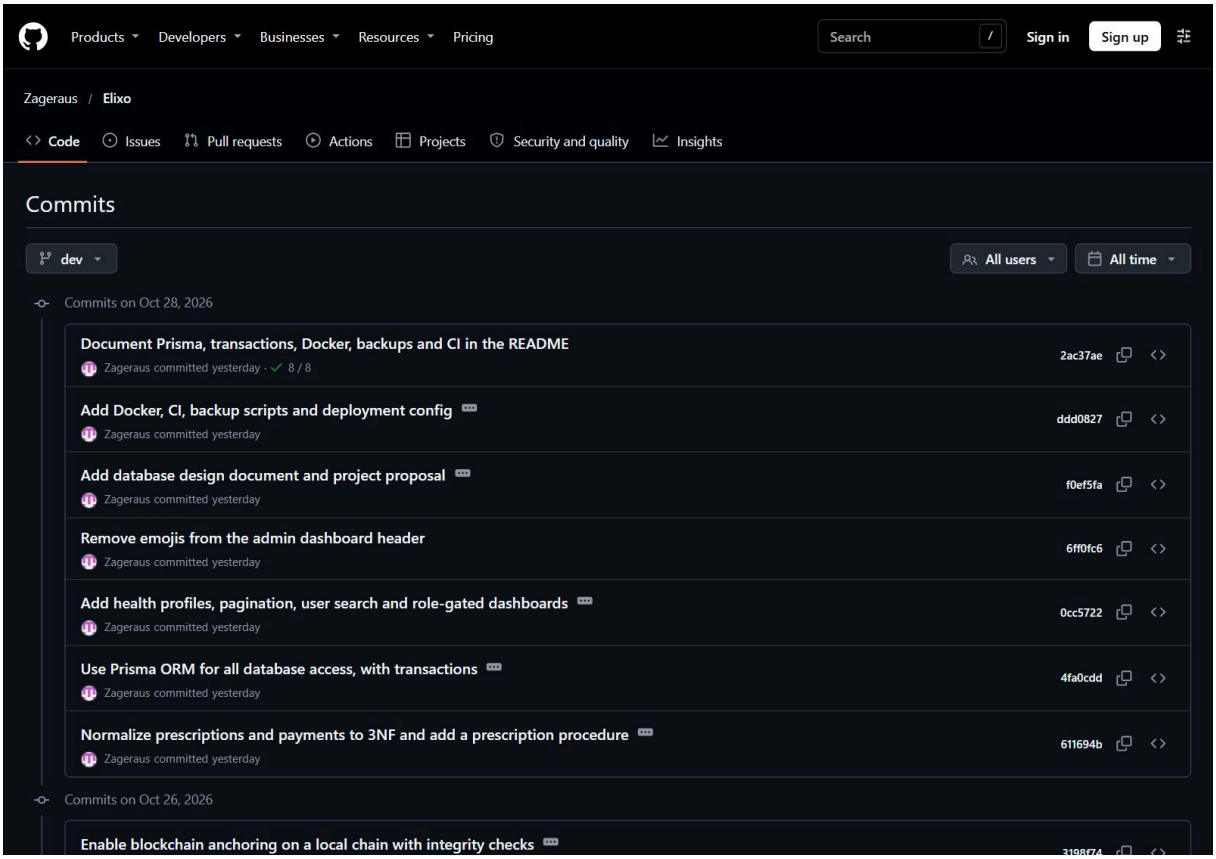


Fig. 13.1 Commits on the dev branch after the review

14. Team Members' Commits

The work was divided into two modules: the database, done by Sanidhya, and the AI and blockchain part, done by Arna. The commits of each member from all our repositories are listed below. The "Repository" column gives the first repository the commit was pushed to (see Section 9.1), and the "Area" column tells which part of the system the commit changes.

Some of Arna's AI and blockchain work (commits 3050504, d458724, 3198f74, 6530d61, 925e4ec, b0e7e8b) was pushed to GitHub from Sanidhya's account, so GitHub shows Zageraus as the author of these commits. They are listed under Arna below, and the "Pushed from" column shows which GitHub account pushed each commit. "Arna*" means the commit was made from a computer where git was set up with a placeholder email (commits f4548fd and d534716), so GitHub does not link it to an account; the author name on both commits is "Arna".

Member	Reg. No	GitHub ID	Module	Commits
Sanidhya Srivastava	24BCB0135	Zageraus	Database	32
Arna Adhikari	24BDS0363	Arnahhh	AI and Blockchain	11

14.1 Sanidhya Srivastava: Database

Sanidhya Srivastava (24BCB0135), GitHub ID Zageraus, Database module: 32 commits.

Repository	Branch	Commit ID	Date	Commit message	Area	Pushed from
HealthChain-AI-and-BlockChain-integrated-	main	2f0ea0f	06 Oct 2026	Initial commit: HealthChain platform	Project setup	Zageraus
HealthChain-AI-and-BlockChain-integrated-	main	705efb5	06 Oct 2026	Add healthchain.zip archive	Project setup	Zageraus
HealthChain-AI-and-BlockChain-integrated-	main	b11eb92	11 Oct 2026	Delete healthchain.zip	Project setup	Zageraus
HealthChain-AI-and-BlockChain-integrated-	main	c41eb12	11 Oct 2026	Update HealthChain codebase with full full-stack architecture, contracts, and AI service	Full stack (schema, API, contracts, AI)	Zageraus
HealthChain-AI-and-BlockChain-integrated-	main	8dda7ab	13 Oct 2026	docs: overhaul README with architecture diagrams, DBMS details, and API reference	Documentation	Zageraus
HealthChain-AI-and-BlockChain-integrated-	main	5d09f0c	13 Oct 2026	feat(frontend): implement doctor prescription issuance form and patient prescription viewer	Frontend	Zageraus
HealthChain-AI-and-BlockChain-integrated-	main	bbedab8	13 Oct 2026	feat(auth): implement admin setup passcode check for admin role registration	Authentication	Zageraus
Elixor-update	main	224db02	20 Oct 2026	Improve Elixor frontend and application	Frontend	Zageraus
Elixo	main	a966cbe	25 Oct 2026	Add email auth, appointment booking, and optional blockchain anchoring	Database / Backend	Zageraus
Elixo	main	7a51624	25 Oct 2026	Guard patient dashboard by role and surface doctor-list errors	Frontend	Zageraus
Elixo	main	9390d47	25 Oct 2026	Sign out automatically when the stored session is rejected	Authentication	Zageraus
Elixo	main	518f977	25 Oct 2026	Fix form reset after booking and report upload	Frontend	Zageraus
Elixo	main	a493dfa	25 Oct 2026	Let doctors accept appointments and scope dashboard data to the user	Database / Backend	Zageraus
Elixo	main	ae28df1	25 Oct 2026	Collect doctor specialization at sign-up and filter doctors by it	Database / Backend	Zageraus
Elixo	main	e6f2c21	25 Oct 2026	Add demo seed data and enable RLS on auth_nonces	Database	Zageraus
Elixo	main	c3a2945	25 Oct 2026	Add simulated consultation payments	Database	Zageraus

Elixo	main	d4ba353	25 Oct 2026	Let admins deactivate or permanently delete patients and doctors	Database / Backend	Zageraus
Elixo	main	6773bd4	25 Oct 2026	Notify patients when their doctor is deactivated or deleted	Database	Zageraus
Elixo	main	1835523	25 Oct 2026	Fix permanent delete of users with anchored records	Database	Zageraus
Elixo	main	c22f815	26 Oct 2026	Make the report upload message reflect what actually happened	Frontend	Zageraus
Elixo	main	4d66d95	26 Oct 2026	Let patients open uploaded report PDFs through signed links	Backend	Zageraus
Elixo	main	174f6b5	26 Oct 2026	Rewrite README to match the current project	Documentation	Zageraus
Elixo	dev	611694b	28 Oct 2026	Normalize prescriptions and payments to 3NF and add a prescription procedure	Database	Zageraus
Elixo	dev	4fa0cdd	28 Oct 2026	Use Prisma ORM for all database access, with transactions	Database	Zageraus
Elixo	dev	0cc5722	28 Oct 2026	Add health profiles, pagination, user search and role-gated dashboards	Database / Frontend	Zageraus
Elixo	dev	6ff0fc6	28 Oct 2026	Remove emojis from the admin dashboard header	Frontend	Zageraus
Elixo	dev	f0ef5fa	28 Oct 2026	Add database design document and project proposal	Database	Zageraus
Elixo	dev	ddd0827	28 Oct 2026	Add Docker, CI, backup scripts and deployment config	DevOps / Backup	Zageraus
Elixo	dev	2ac37ae	28 Oct 2026	Document Prisma, transactions, Docker, backups and CI in the README	Documentation	Zageraus
Elixo	dev	7bfab2a	29 Oct 2026	Return 409 when registering an email that is already in use	Backend	Zageraus
Elixo	dev	18918c0	29 Oct 2026	Add a plain-language project overview PDF	Documentation	Zageraus
Elixo	dev	3959539	30 Oct 2026	Listen on 0.0.0.0 for Render and ignore the .vercel folder	Deployment	Zageraus

14.2 Arna Adhikari: AI and Blockchain

Arna Adhikari (24BDS0363), GitHub ID Arnahhh, AI and Blockchain module: 11 commits.

Repository	Branch	Commit ID	Date	Commit message	Area	Pushed from

HealthChain-Finalised	main	`f4548fd`	16 Oct 2026	Initial HealthChain project upload	Full project (incl. AI service, contracts)	Arna*
HealthChain-AI-and-BlockChain-integrated-	healthchain-upgrade	`d534716`	18 Oct 2026	Upgrade HealthChain AI and blockchain integration	AI and Blockchain	Arna*
HealthChain-AI-and-BlockChain-integrated-	main	`ba54fd3`	18 Oct 2026	Merge pull request #1 from Zageraus/healthchain-upgrade	AI and Blockchain	Arnahhh
HealthChain-AI-and-BlockChain-integrated-	main	`ff5fd13`	19 Oct 2026	Add local development files	Project setup	Arnahhh
Elixor-update	main	`8abbccf`	21 Oct 2026	Prepare ELIXOR project update	Frontend / Authentication	Arnahhh
Elixo	main	`3050504`	25 Oct 2026	Show names on prescriptions and only badge truly anchored ones	Blockchain	Zageraus
Elixo	main	`d458724`	26 Oct 2026	Show the real anchoring status on report cards	Blockchain	Zageraus
Elixo	main	`3198f74`	26 Oct 2026	Enable blockchain anchoring on a local chain with integrity checks	Blockchain	Zageraus
Elixo	dev	`6530d61`	30 Oct 2026	Use Claude for report summaries on the deployed server	AI	Zageraus
Elixo	dev	`925e4ec`	30 Oct 2026	Add a free OpenAI-compatible AI provider (Groq) for live summaries	AI	Zageraus
Elixo	dev	`b0e7e8b`	30 Oct 2026	Pick an available AI model when the configured one is gone	AI	Zageraus

15. Challenges Faced

Some of the problems we ran into while doing this project, and how we solved them:

1. **Normalization mistakes in our first design.** We had stored the medicines of a prescription as a JSON list, and the payments table had patient and doctor columns. After the first review we understood that these break 1NF and 3NF. Fixing them meant writing a migration that moved the old data into the new prescription_items table without losing anything, and then updating the backend and frontend to use the new table.
2. **Moving to an ORM.** We started with the Supabase query builder. Changing every query to Prisma took a lot of time, and Prisma cannot read views or call procedures directly, so for those we had to use `$queryRaw`.
3. **Keeping data consistent.** Earlier, when one step of an operation failed (for example creating a doctor's profile after creating the user), we had to delete the half created data

ourselves. Using transactions solved this, but we had to restructure the code so that all the steps run inside one transaction.

4. **Database connection timeouts.** The Supabase connection sometimes timed out on slow internet. We fixed this by increasing the connect and pool timeouts in the database URL.
5. **Recording who made a change.** Triggers run inside the database, so they did not know which app user made the change. We set `app.current_user_id` at the start of each transaction so that the audit trigger can read it.
6. **Small bugs found while testing.** For example, registering with an email that was already used gave a 400 error instead of 409. We found it while preparing this report and fixed it.
7. **AI and blockchain on the live server.** After deploying, report uploads on the live site showed "AI summary failed: fetch failed" and "Can't reach the blockchain node". The backend was still trying to reach Ollama and the Hardhat chain on its own machine, which only exist on our laptops. We turned anchoring off on the live server. For AI we first added Claude, but it is a paid API, so we switched to Groq's free API. The first model we chose was not available for our Groq key, so we made the backend pick an available model on its own (Section 10).

16. URLs, Tools and Resources

Resource	Link
GitHub repository	https://github.com/Zageraus/Elixo
Commit history	https://github.com/Zageraus/Elixo/commits/main
CI runs	https://github.com/Zageraus/Elixo/actions
Database design document	https://github.com/Zageraus/Elixo/blob/main/docs/DATABASE_DESIGN.md
Project proposal	https://github.com/Zageraus/Elixo/blob/main/docs/PROJECT_PROPOSAL.md
SQL migrations	https://github.com/Zageraus/Elixo/tree/main/db/migrations
Live application	https://elixor-dun.vercel.app
Live API (health check)	https://elixor-backend.onrender.com/health
Local setup	http://localhost:3000 (frontend), http://localhost:4000/health (API)

Tool / library	Version	Link
PostgreSQL on Supabase	17	https://supabase.com
Prisma ORM	6.19	https://www.prisma.io
Express.js	4.19	https://expressjs.com
Next.js / React	14.2 / 18.3	https://nextjs.org

Tailwind CSS	3.4	https://tailwindcss.com
Zod	3.23	https://zod.dev
jsonwebtoken	9.0	https://github.com/auth0/node-jsonwebtoken
Groq API (free tier)	openai/gpt-oss-120b	https://console.groq.com
Ollama	-	https://ollama.com
Hardhat / Solidity	- / 0.8.24	https://hardhat.org
Docker	-	https://www.docker.com
GitHub Actions	-	https://docs.github.com/actions
Render, Vercel	-	https://render.com, https://vercel.com

APIs: our own REST API (the endpoints are listed in the README), Supabase Auth and Storage, the Groq chat completions API, the Ollama local API and an Ethereum JSON-RPC node (Hardhat) for anchoring.

Dataset: we did not use any external dataset. The sample data (doctors, patients, appointments, prescriptions, reports) was written by us and is in `db/seed/`. It does not have any real patient information.

17. Conclusion and Future Work

17.1 Conclusion

In this project we built a working clinic management system on top of a properly designed relational database. The most useful things we learned were how much of the business logic can be put in the database itself using constraints, triggers and a stored procedure, why normalization matters in practice (we found two real problems in our own first design), and how transactions keep the data correct when many tables are changed together. We also got hands-on experience with an ORM, Git branching, Docker and CI, which we had not used much before this course.

17.2 Future Work

- Adding a real payment gateway in test mode.
- Reading the text from uploaded PDF reports using OCR.
- A verification step for new doctor accounts.
- Deploying the smart contract on a public test network.
- Storing the login token in an httpOnly cookie instead of browser storage.

REFERENCES

1. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
2. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
3. PostgreSQL Global Development Group, "PostgreSQL 17 Documentation." https://www.postgresql.org/docs/17/
4. Prisma, "Prisma ORM Documentation." https://www.prisma.io/docs
5. Supabase, "Supabase Documentation." https://supabase.com/docs
6. Vercel, "Next.js Documentation." https://nextjs.org/docs
7. OpenJS Foundation, "Express.js Documentation." https://expressjs.com
8. Groq, "GroqCloud Documentation." https://console.groq.com/docs
9. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
10. GitHub, "GitHub Actions Documentation." https://docs.github.com/actions